



Systemes d'information avec PHP et MySQL

Bruno Bachelet

bruno@nawouak.net

<http://www.nawouak.net>



PARTIE I

PHP: script côté serveur

Copyright (c) 1999-2011 - Bruno Bachelet - bruno@nawouak.net - <http://www.nawouak.net>

La permission est accordée de copier, distribuer et/ou modifier ce document sous les termes de la licence *GNU Free Documentation License*, Version 1.1 ou toute version ultérieure publiée par la fondation *Free Software Foundation*. Voir cette licence pour plus de détails (<http://www.gnu.org>).



- HTML n'est pas un langage de programmation
 - Langage de structuration de l'information
 - Tendence actuelle: séparer l'information de son apparence
 - HTML contient l'information seulement
 - Feuille de style (CSS) contient la présentation de l'information

- Avantages de HTML
 - Développement facilité
 - Intégration multimédia
 - Texte, images, liens, vidéo...

- Inconvénients de HTML
 - Présentation statique de l'information
 - Pas d'interaction avec l'utilisateur



Scripts et pages dynamiques

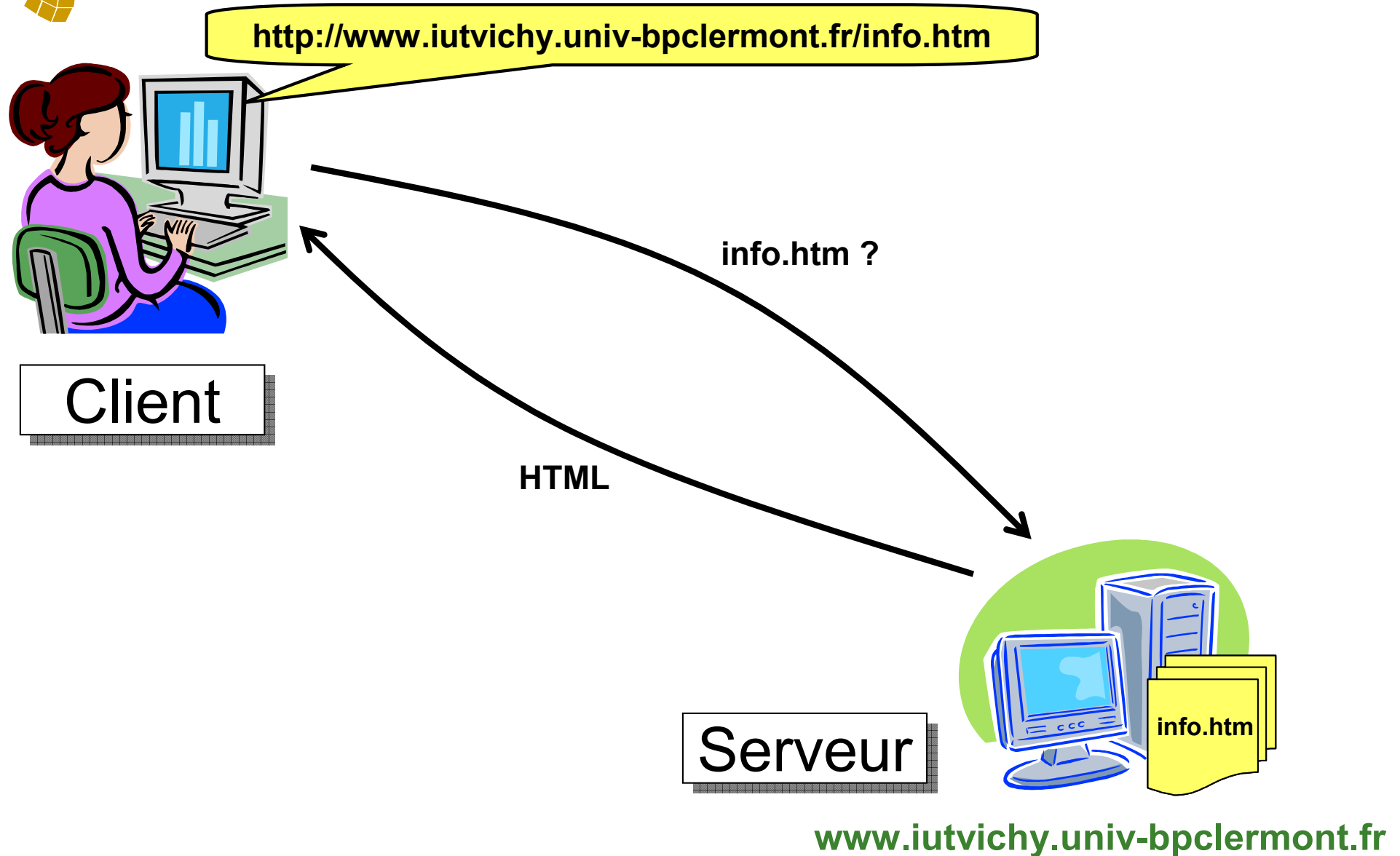
- Les scripts côté client (comme JavaScript) rendent les pages dynamiques
 - Modification dynamique de la présentation du contenu
 - Réponse aux actions de l'utilisateur
 - Vérification des formulaires
 - ...

- Ils répondent au besoin d'interactivité
 - Mais pas au besoin de dynamisme du contenu

- D'où la nécessité de scripts côté serveur (comme PHP)
 - Permettent la modification dynamique du contenu
 - Interagissent avec des bases de données



Client-serveur: HTML simple (1/2)





Client-serveur: HTML simple (2/2)

- Sur le serveur: code HTML

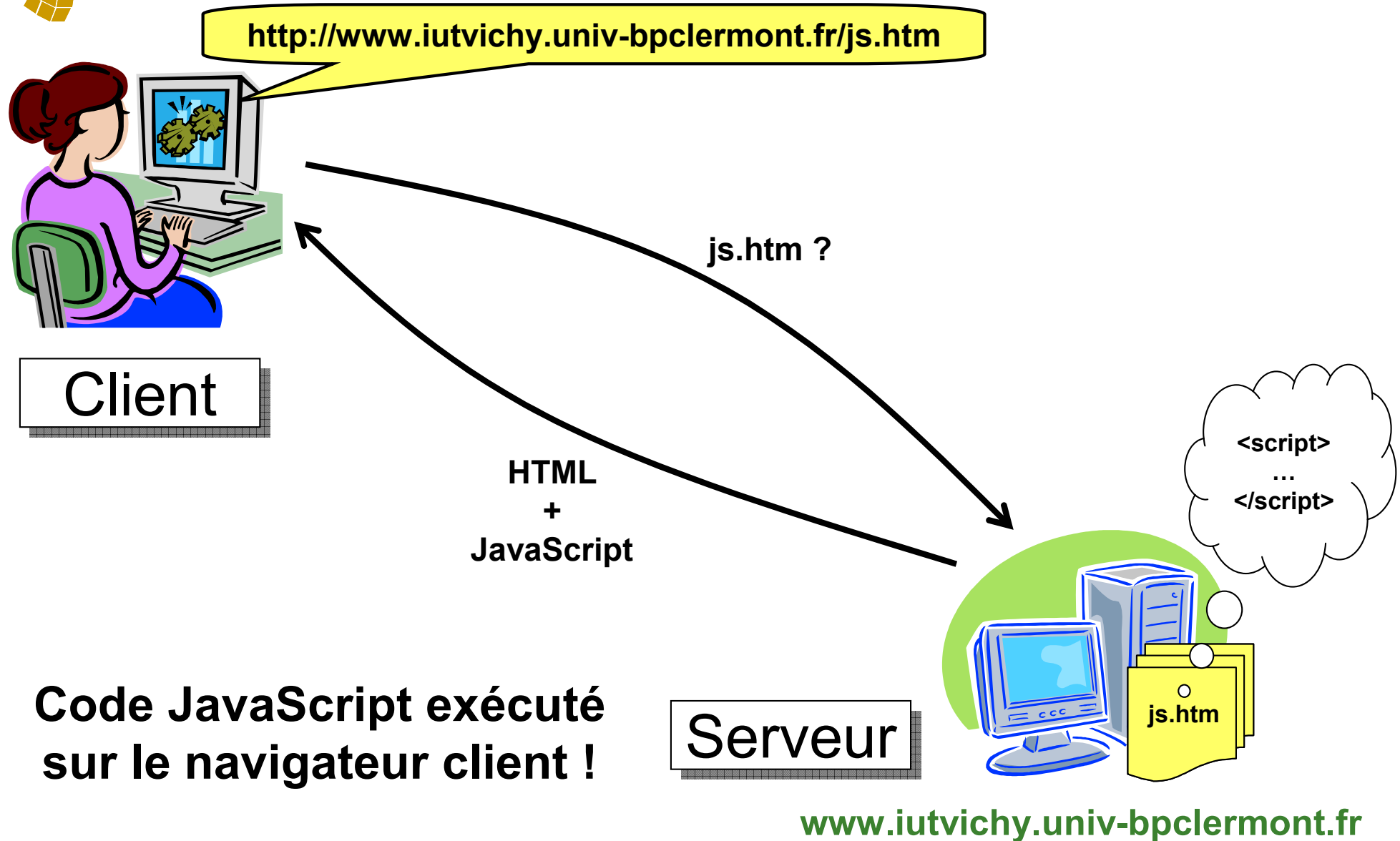
```
<html>
  <body>
    <p>Bienvenue !</p>
  </body>
</html>
```

- Le client reçoit: code HTML identique

```
<html>
  <body>
    <p>Bienvenue !</p>
  </body>
</html>
```



Client-serveur: code JavaScript (1/2)





Client-serveur: code JavaScript (2/2)

- Sur le serveur: code HTML + JavaScript

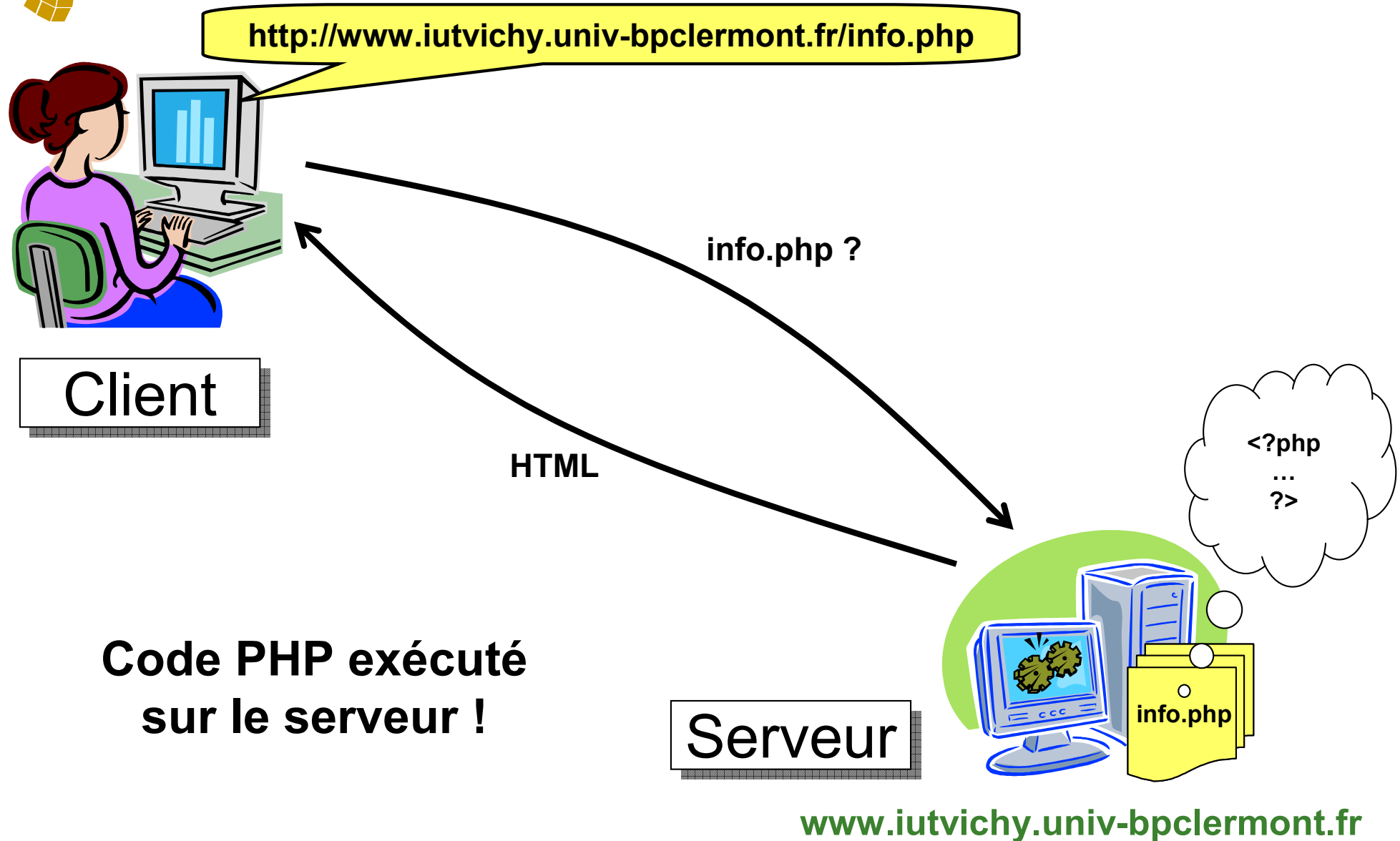
```
<html>
  <body>
    <script type="text/javascript">
      document.write("<p>Bienvenue !</p>");
    </script>
  </body>
</html>
```
- Le client reçoit: code HTML + JavaScript identique

```
<html>
  <body>
    <script type="text/javascript">
      document.write("<p>Bienvenue !</p>");
    </script>
  </body>
</html>
```
- Le navigateur affiche: code HTML

```
<html>
  <body>
    <p>Bienvenue !</p>
  </body>
</html>
```




Client-serveur: code PHP (1/2)





Client-serveur: code PHP (2/2)

- Sur le serveur: code PHP

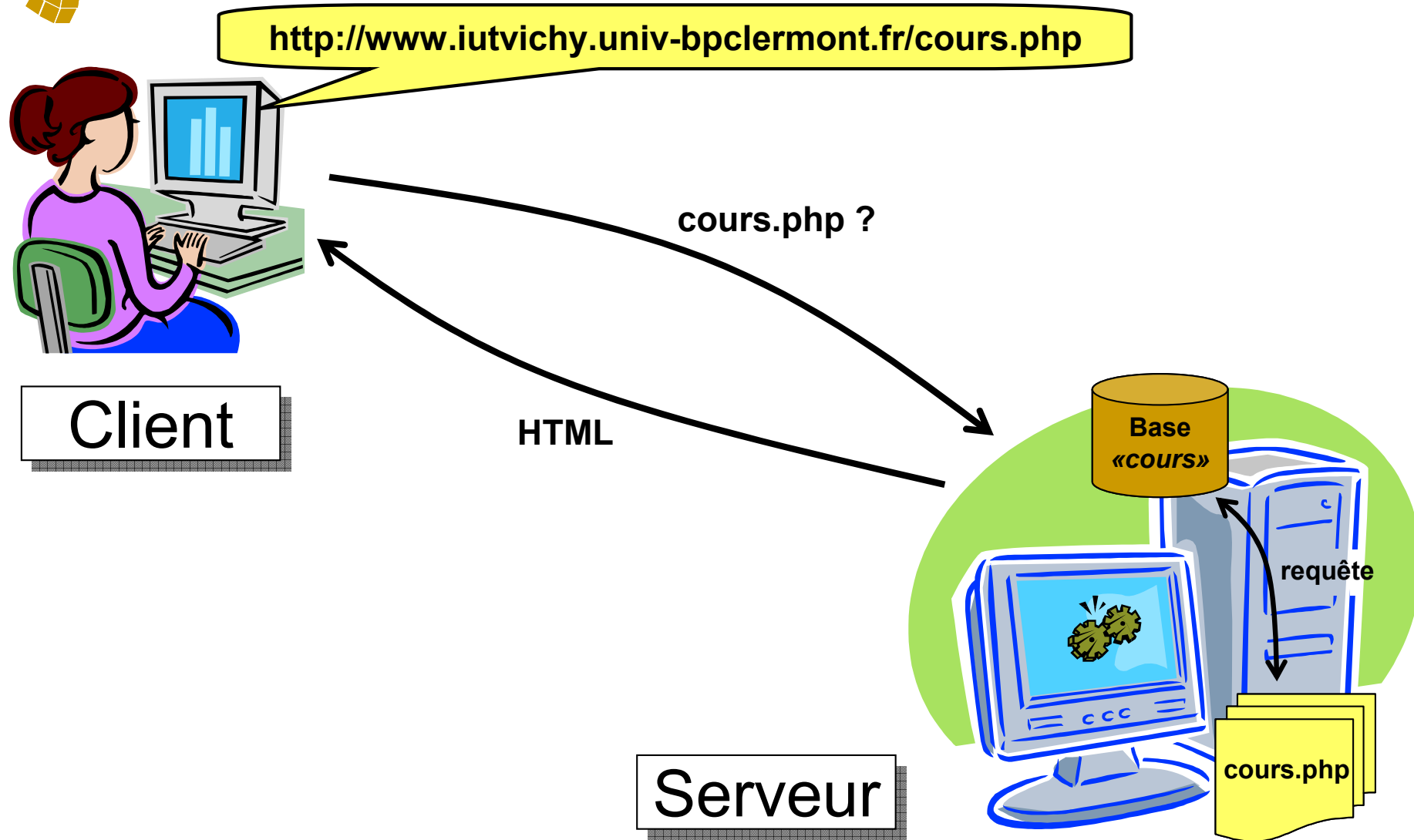
```
<html>
  <body>
    <?php
      echo "<p>Bienvenue !</p>";
    ?>
  </body>
</html>
```

- Le client reçoit: code HTML

```
<html>
  <body>
    <p>Bienvenue !</p>
  </body>
</html>
```



Client-serveur: base de données (1/2)



www.iutvichy.univ-bpclermont.fr



Client-serveur: base de données (2/2)

- Sur le serveur: code PHP

```
<html>
<body>
  <?php
    ... @mysql_query("SELECT * FROM cours"); ... // Requête

    while ... {
      ... echo "<p>$cours</p>"; ... // Affichage liste
    }
  ?>
</body>
</html>
```

- Le client reçoit: code HTML

```
<html>
<body>
  ...
  <p>Systèmes d'information</p>
  <p>Programmation objet et événementielle</p>
  ...
</body>
</html>
```



JavaScript et PHP sont complémentaires

■ JavaScript

- ❑ Exécuté côté client
- ❑ Code lisible par l'utilisateur
- ❑ Permet de modifier la structure de la page
 - Interactivité forte avec l'utilisateur
- ❑ Peut fonctionner «hors ligne»

■ PHP

- ❑ Exécuté côté serveur
- ❑ Code non lisible par l'utilisateur
- ❑ Ne permet pas de modifier la structure de la page
 - Interactivité plus difficile avec l'utilisateur
 - Rechargement (contact avec le serveur) nécessaire
- ❑ Accès à une base de données
 - Enregistrement et restitution de données
- ❑ Ne peut pas fonctionner «hors ligne»



- PHP permet d'accéder à des bases de données
 - Différents types de bases sont possibles
 - Par exemple: MySQL, PostgreSQL

- PHP nécessite un serveur pour fonctionner
 - Par exemple: serveur LAMP, WAMP ou MAMP
 - Linux, Windows ou Mac OS: système d'exploitation
 - Apache: serveur HTTP
 - MySQL: gestionnaire bases de données
 - PHP: interpréteur script

- Solution simple sous Windows: **EasyPHP**



PARTIE II

Le langage PHP

Copyright (c) 1999-2011 - Bruno Bachelet - bruno@nawouak.net - <http://www.nawouak.net>

La permission est accordée de copier, distribuer et/ou modifier ce document sous les termes de la licence *GNU Free Documentation License*, Version 1.1 ou toute version ultérieure publiée par la fondation *Free Software Foundation*. Voir cette licence pour plus de détails (<http://www.gnu.org>).



- 1994: création du langage par Rasmus Lerdorf
 - Pour sa page personnelle
 - PHP = *P*ersonal *H*ome *P*age

- 1997: PHP 3
 - PHP est devenu un projet collectif
 - PHP = *P*HP: *H*ypertext *P*reprocessor
(Acronyme récursif comme LINUX)

- 2001: PHP 4

- 2004: PHP 5



Page statique / page dynamique

- Page HTML = page statique
 - ❑ Extension « **.htm** » ou « **.html** »
 - ❑ Transmise directement au client
 - ❑ Code source lisible par le client

- Page PHP = page dynamique
 - ❑ Extension « **.php** », « **.php3** » ou « **.php4** »
 - Conseil: utiliser plutôt l'extension « **.php** »
 - ❑ Interprétée par le serveur
 - ❑ Génère du HTML transmis au client
 - ❑ Code source non lisible par le client



- Code PHP inséré dans code HTML

- Balise de début: `<?php`
- Balise de fin: `?>`

- Exemple très simple

```
<html>
  <head>
    <title>Hello World !</title>
  </head>

  <body>
    <?php
      echo "Hello World !";
    ?>
  </body>
</html>
```



Génération de code HTML

- «**echo**» permet de générer du HTML
 - Equivalent de «**document.write**» en JavaScript

- Côté serveur: code PHP + HTML

```
<html>
  <body>
    <?php echo "<b>Hello World !</b>"; ?>
  </body>
</html>
```

- Côté client: code HTML

```
<html>
  <body>
    <b>Hello World !</b>
  </body>
</html>
```



- Syntaxe similaire à Java
 - Instructions terminées par « ; »
 - Commentaires
 - « // » pour une seule ligne
 - « /* ... */ » pour plusieurs lignes
 - Opérateurs identiques
 - Structures de contrôle identiques
 - Sensible à la casse

- Nom des variables précédé d'un « \$ »

- Variables faiblement typées
 - Pas de type prédéfini
 - Type peut changer en cours d'exécution

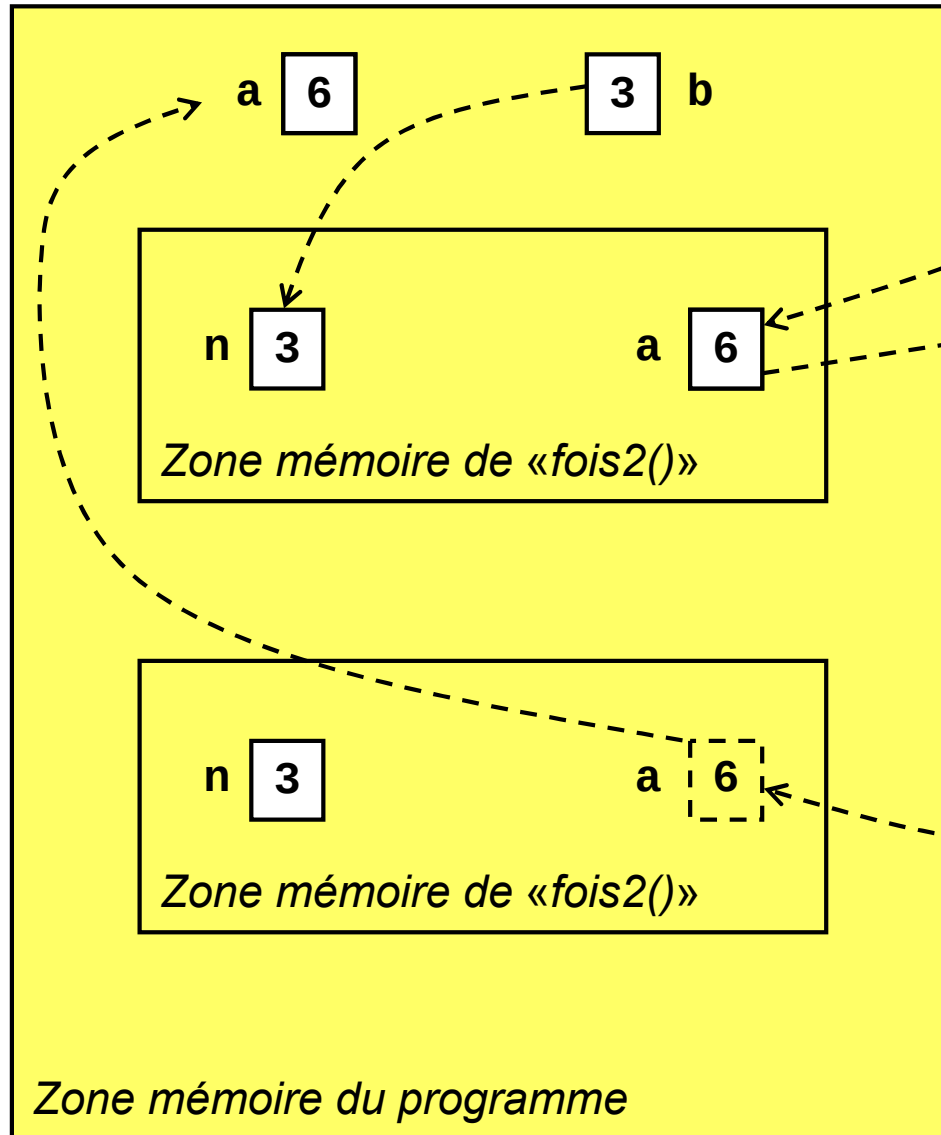


- Déclaration implicite
 - ❑ `$nom_variable = valeur;`
 - ❑ Aucun mot-clé particulier
 - ❑ Déclarée à la première utilisation
 - ❑ Portée de la variable limitée au bloc d'instructions
 - Bloc d'instructions = zone entre «`{`» et «`}`»
 - En dehors de tout bloc = variable globale
 - Dans le bloc d'une fonction = variable locale

- Pour utiliser une variable globale
 - ❑ Mot-clé «`global`»
 - ❑ `global $nom_variable`



Variables (2/2)



- Exemple de manipulation de variables

```
$a = 7;  
$b = 3;
```

```
function fois2($n) {  
    $a = 2 * $n;  
    return $a;  
}
```

```
echo "Double de ".$b." = ".fois2($b);  
echo "Valeur de a = ".$a;
```

- Résultat de l'exemple

```
Double de 3 = 6  
Valeur de a = 7
```

- Modification de la fonction

```
function fois2($n) {  
    global $a;  
    $a = 2 * $n;  
    return $a;  
}
```

- Résultat avec la modification

```
Double de 3 = 6  
Valeur de a = 6
```



- Type déterminé à l'initialisation de la variable
 - Booléen
 - `$x = TRUE;`
 - `$x = FALSE;`
 - Entier
 - `$x = 27;`
 - Flottant
 - `$x = 27.13;`
 - Chaîne de caractères
 - `$x = "Nawouak";`
 - `$x = "27";`

- Le type d'une variable peut changer en cours d'exécution
 - `$x = 5;`
 - `...`
 - `$x = "cinq";`



■ Opérateurs classiques

- Arithmétique: **+**, **-**, *****, **/**, **%**
- Affectation: **=**, **+=**, **-=**, ***=**, **/=**, **%=**, **--**, **++**
- Logique: **&&** (ou **and**), **||** (ou **or**), **!**
- Comparaison: **==**, **<**, **>**, **<=**, **>=**, **!=**

■ Test sur les variables

- **isset(\$var)**
 - «**TRUE**» si «**\$var**» existe
 - «**FALSE**» sinon
- **empty(\$var)**
 - «**TRUE**» si «**\$var**» n'existe pas ou nulle (zéro, chaîne vide)
 - «**FALSE**» sinon



Structures de contrôle (1/4)

- Bloc d'instructions délimité par «{» et «}»
 - Facultatif si une seule ligne
 - `if (x == 1) y = 2;` $\Leftrightarrow$ `if (x == 1) { y = 2; }`

- Structures conditionnelles
 - `if (test1) action1`
 `else if (test2) action2`
 `else action3`

 - `switch (variable)`
 `case valeur1: action1`
 `case valeur2: action2`
 `default: action3`

- Structures de boucle
 - `while (test) action`
 - `do action while (test)`
 - `for (depart; test; suivant) action`



Structures de contrôle (2/4)

- *"une année bissextile est un multiple de 4, mais pas de 100, mais de 400"*

- Version 1

```
if ($annee % 4 != 0) { $bissextile = FALSE; }  
else if ($annee % 100 != 0) { $bissextile = TRUE; }  
else if ($annee % 400 != 0) { $bissextile = FALSE; }  
else { $bissextile = TRUE; }
```

- Version 2

```
if ($annee % 4 == 0) {  
    if ($annee % 100 == 0) {  
        if ($annee % 400 == 0) ✗ $bissextile = TRUE; ✗  
        else ✗ $bissextile = FALSE; ✗  
    }  
    else ✗ $bissextile = TRUE; ✗  
}  
else ✗ $bissextile = FALSE; ✗
```



Structures de contrôle (3/4)

- Exemple: conversion d'un chiffre en lettres

- Version avec «**if**» imbriqués

```
if ($x == 1) $texte = "un";  
else if ($x == 2) $texte = "deux";  
...  
else if ($x == 9) $texte = "neuf";  
else $texte = "zéro";
```

- Version avec «**switch**»

```
switch ($x) {  
    case 1: $texte = "un"; break;  
    case 2: $texte = "deux"; break;  
    ...  
    case 9: $texte = "neuf"; break;  
    default: $texte = "zéro";  
}
```



Structures de contrôle (4/4)

- Exemple: calcul de $9! = 9 \times 8 \times \dots \times 2 \times 1$

- Calcul avec boucle «**for**»

```
$f = 1;
```

```
for ($i = 2; $i < 10; $i++) { $f *= $i; }
```

- Calcul avec boucle «**while**»

```
$f = 1;
```

```
$i = 2;
```

```
while ($i < 10) { $f *= $i; $i++; }
```

- Calcul avec boucle «**do...while**»

```
$f = 1;
```

```
$i = 2;
```

```
do { $f *= $i; $i++; } while ($i < 10);
```



Chaînes de caractères (1/3)

- Délimiteurs et concaténation
 - Variables non évaluées avec « ' »
`'salut $nom'` $\Rightarrow$ `salut $nom`
 - Variables évaluées avec « " »
`"salut $nom"` $\Rightarrow$ `salut Bruno`
`"${formation}2007"` $\Rightarrow$ `src2007`
 - Opérateur de concaténation
`"salut ".$nom` $\Rightarrow$ `salut Bruno`
- Symbole \
 - `\$` $\Rightarrow$ `$`
 - `\\` $\Rightarrow$ `\`
 - `\"` $\Rightarrow$ `"`
 - `\'` $\Rightarrow$ `'`
 - `\n` $\Rightarrow$ nouvelle ligne



Chaînes de caractères (2/3)

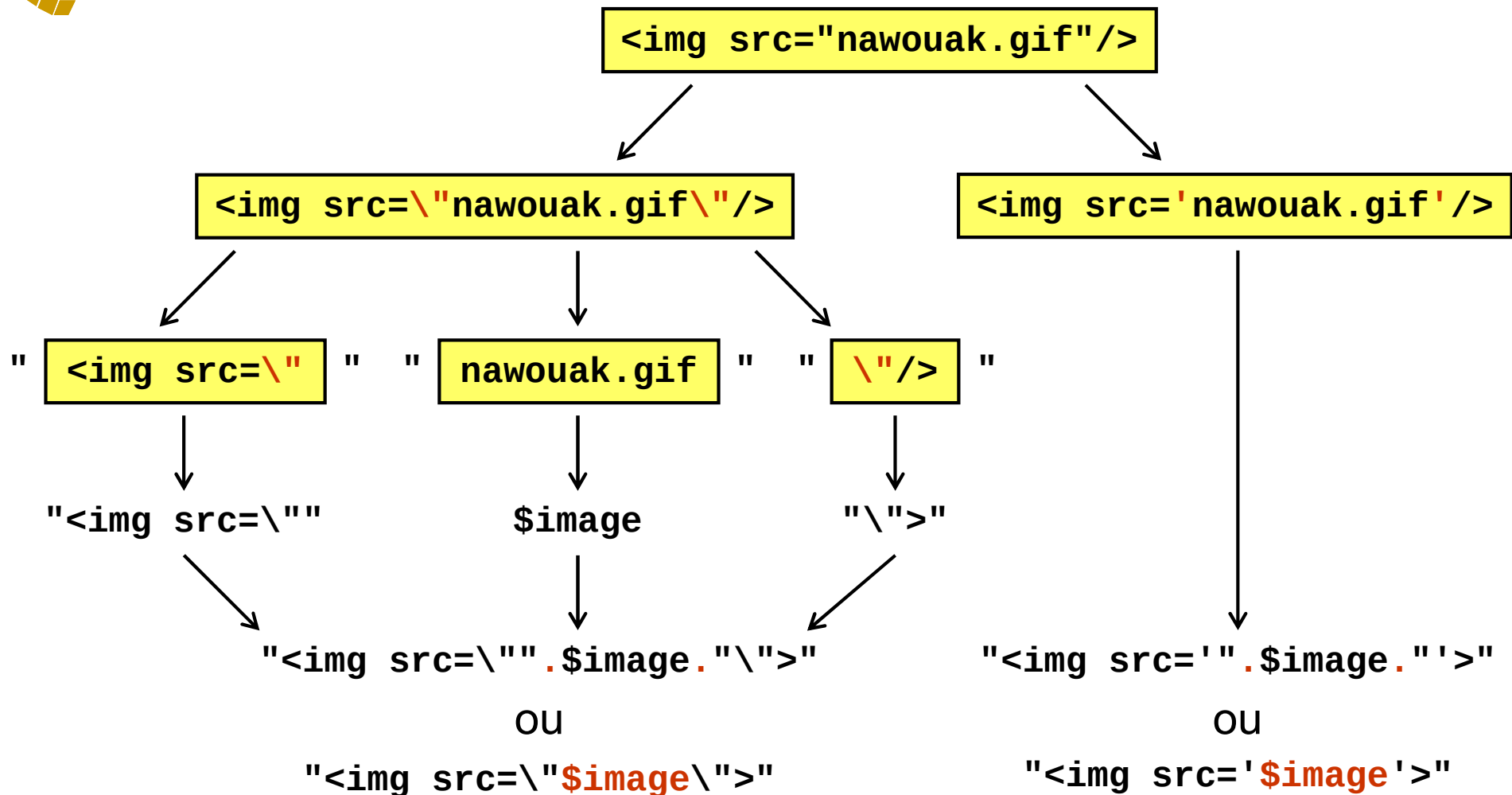
- Comment mettre un délimiteur dans une chaîne ?
 - Problème identique en JavaScript

- Exemple: générer le code HTML d'une image
 - Code HTML à générer
 - ``
 - Nom de l'image dans une variable
 - `$image = "nawouak.gif";`

- Plusieurs possibilités
 - Opérateur de concaténation
 - Evaluation de «`$image`» dans chaîne
 - Pas d'équivalent en JavaScript
 - Choix du délimiteur «`'`» ou «`"`»
 - Alternance des délimiteurs «`'`» et «`"`»



Chaînes de caractères (3/3)



Même logique avec « ' », mais attention « '\$image' » ne fonctionne pas !



Tableaux simples (1/2)

- La déclaration du tableau n'est pas nécessaire
- Le remplissage du tableau n'est pas forcément continu

```
$menu[3] = "CV";  
$menu[1] = "Accueil";  
$menu[7] = "Photos";
```
- Les indices sont facultatifs, la numérotation commence à 0

```
$menu[] = "Accueil";  
$menu[] = "CV";  
$menu[] = "Photos";
```
- Déclaration accélérée
 - `$menu = array("Accueil", "CV", "Photos");`
- Connaître le nombre d'éléments
 - `$nb = count($menu);`



■ Parcours des éléments

❑ Code PHP

```
for ($i = 0; $i < count($menu); $i++) {  
    echo "Rubrique $i: $menu[$i]<br/>";  
}
```

❑ Code HTML généré

```
Rubrique 0: Accueil<br/>  
Rubrique 1: CV<br/>  
Rubrique 2: Photos<br/>
```

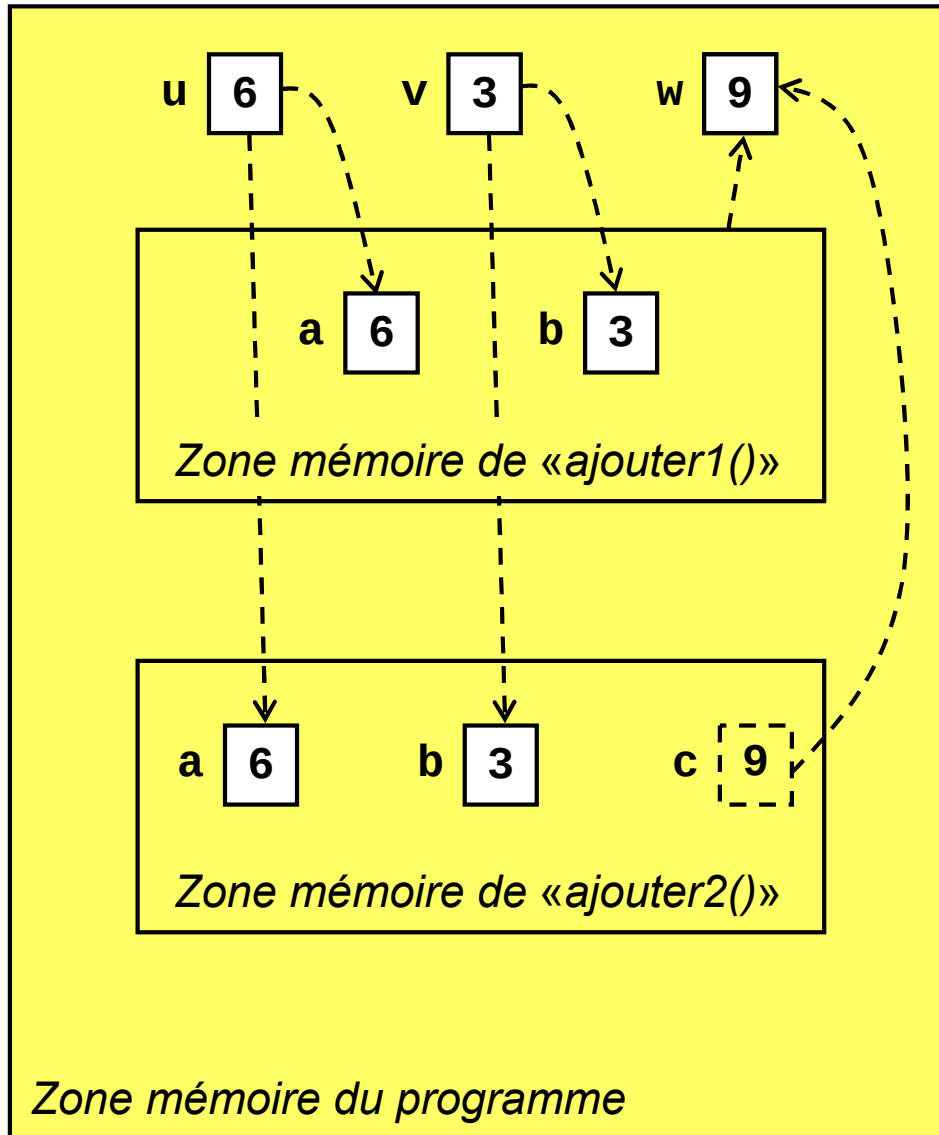
■ Uniquement pour un remplissage continu !



- Permettent de factoriser / réutiliser du code
 - ❑ Portions de code associées à un nom
 - ❑ Reçoivent des arguments en entrée
 - ❑ Retournent une valeur en sortie
- **function** *nom(arguments)* {
 action;
 return *resultat*;
}
- Exemple
function *ajouter(\$a,\$b)* {
 \$c = **\$a** + **\$b**;
 return **\$c**;
}



Fonctions (2/3)



- Passage par valeur
 - ❑ `function f($x)`
 - ❑ L'argument «`$x`» est une copie
 - ❑ Modification limitée à «`f`»
 - ❑

```
function ajouter1($a,$b)
{ return ($a + $b); }
```
 - ❑ `$w = ajouter1($u,$v);`
- Passage par référence
 - ❑ `function f(&$x)`
 - ❑ L'argument «`$x`» est un alias
 - ❑ Modification répercutée hors de «`f`»
 - ❑

```
function ajouter2($a,$b,&$c)
{ $c = $a + $b; }
```
 - ❑ `ajouter2($u,$v,$w);`
- A l'appel, aucune différence
`y = f($x);`



- Possibilité de rassembler des fonctions dans un fichier
 - «**include**» inclut et exécute un fichier
 - Alerte en cas d'absence
 - «**require**» inclut et exécute un fichier
 - Erreur en cas d'absence
- Exemple
 - Fichier «**fonctions.php**» contenant une fonction

```
<?php
...
function ajouter($a,$b) { return ($a + $b); }
...
?>
```
 - Utilisation du fichier et de la fonction

```
<?php
include("fonctions.php");
...
echo "Somme = ".ajouter(5,13);
...
?>
```



Tableaux associatifs (1/3)

- Tableaux indexés par des chaînes
 - Association d'une clé (chaîne) à un élément
 - Tableau simple: association entier-élément

- Initialisation d'un tableau
 - ```
$menu["home"] = "Accueil";
$menu["cv"] = "CV";
$menu["photos"] = "Photos";
```
  
  - ```
$menu = array(  
    "home" => "Accueil",  
    "cv" => "CV",  
    "photos" => "Photos");
```



- Parcours des éléments

- Code PHP

```
foreach ($menu as $element) {  
    echo "Rubrique \"$element\"<br/>";  
}
```

- Code HTML généré

```
Rubrique "Accueil"<br/>  
Rubrique "CV"<br/>  
Rubrique "Photos"<br/>
```

- Fonctionne aussi pour des tableaux simples !



■ Parcours des couples clé-élément

□ Code PHP

```
foreach ($menu as $cle => $element) {  
    echo "Rubrique <a href='$cle.htm'>";  
    echo "$element</a><br/>";  
}
```

□ Code HTML généré

```
Rubrique <a href='home.htm'>Accueil</a><br/>  
Rubrique <a href='cv.htm'>CV</a><br/>  
Rubrique <a href='photos.htm'>Photos</a><br/>
```



PARTIE III

PHP et MySQL

Copyright (c) 1999-2011 - Bruno Bachelet - bruno@nawouak.net - <http://www.nawouak.net>

La permission est accordée de copier, distribuer et/ou modifier ce document sous les termes de la licence *GNU Free Documentation License*, Version 1.1 ou toute version ultérieure publiée par la fondation *Free Software Foundation*. Voir cette licence pour plus de détails (<http://www.gnu.org>).



PHP et les bases de données

- SQL: *Structured Query Language*
 - ❑ Langage universel des bases de données
 - ❑ Permet l'interrogation d'une base
 - ❑ Permet la modification d'une base

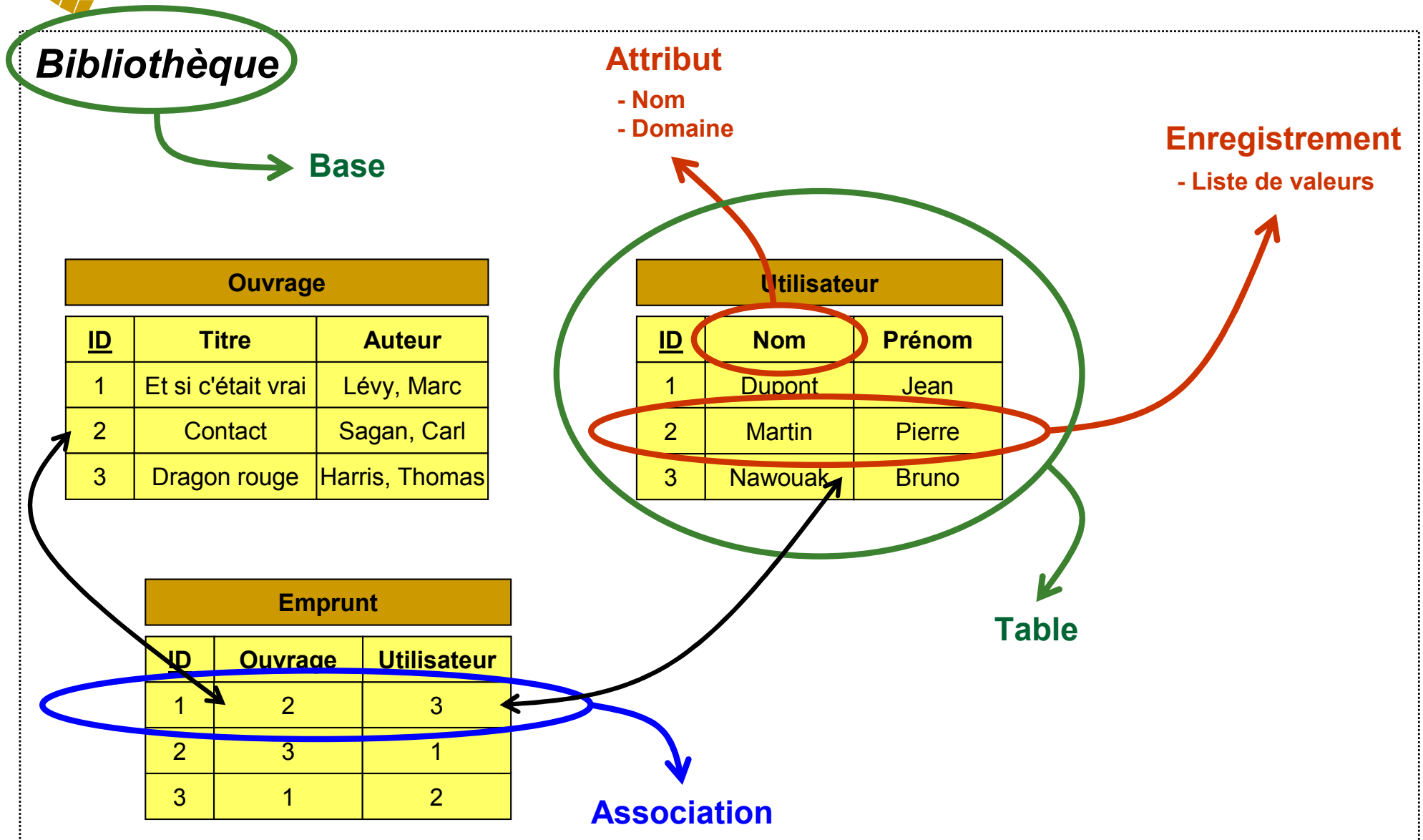
- MySQL
 - ❑ Gestionnaire de bases de données
 - ❑ Commandé par le langage SQL

- PHP peut interagir avec MySQL
 - ❑ Envoi de requêtes SQL
 - ❑ Récupération du résultat
 - ❑ Ajout de données

- phpMyAdmin
 - ❑ Interface Web pour accéder à MySQL
 - ❑ Facilite la gestion des bases
 - ❑ Permet de tester ses requêtes



Bases de données: vocabulaire





Types d'attributs particuliers

- Attribut en clé primaire
 - ❑ Identifiant unique d'un enregistrement
 - ❑ Accès direct à l'enregistrement par la clé
 - ❑ Accélère les jointures où la clé sert de pivot

- Attribut en index
 - ❑ N'est pas forcément unique dans la table
 - ❑ Accélère les tris (ou recherches) sur l'attribut
 - Sans index: recherche séquentielle
 - Avec index: table organisée en zones



Requêtes SQL: projection

- Sélection de certains attributs d'une table
- **SELECT titre, auteur FROM ouvrage**

Ouvrage		
<u>ID</u>	Titre	Auteur
1	Et si c'était vrai	Lévy, Marc
2	Contact	Sagan, Carl
3	Dragon rouge	Harris, Thomas
4	Où es-tu ?	Lévy, Marc
5	Hannibal	Harris, Thomas



Titre	Auteur
Et si c'était vrai	Lévy, Marc
Contact	Sagan, Carl
Dragon rouge	Harris, Thomas
Où es-tu ?	Lévy, Marc
Hannibal	Harris, Thomas

- « * » signifie tous les attributs
 - **SELECT * FROM ouvrage**



Requêtes SQL: sélection (1/2)

- Sélection d'enregistrements d'une table vérifiant certaines conditions
- **SELECT * FROM ouvrage
WHERE auteur='Lévy, Marc'**

Ouvrage		
<u>ID</u>	Titre	Auteur
1	Et si c'était vrai	Lévy, Marc
2	Contact	Sagan, Carl
3	Dragon rouge	Harris, Thomas
4	Où es-tu ?	Lévy, Marc
5	Hannibal	Harris, Thomas

Sélection →

<u>ID</u>	Titre	Auteur
1	Et si c'était vrai	Lévy, Marc
4	Où es-tu ?	Lévy, Marc



Requêtes SQL: sélection (2/2)

- Les conditions peuvent être combinées
 - Avec «**AND**» et «**OR**»
 - **SELECT * FROM ouvrage**
WHERE (auteur='Lévy, Marc'
OR auteur='Harris, Thomas')

- Possibilité de filtre
 - «**%**» = caractères joker
 - **SELECT * FROM ouvrage**
WHERE (auteur LIKE '%Marc%')
 - Tous les auteurs avec «*Marc*» dans le nom
 - **SELECT * FROM ouvrage WHERE (auteur LIKE 'H%')**
 - Tous les auteurs avec un nom commençant par «*H*»



Requêtes SQL: tri (1/2)

- A la suite d'une requête: mot-clé «**ORDER BY**»
- **SELECT * FROM utilisateur ORDER BY nom, prenom**
 - ❑ Tri sur le premier attribut («*nom*»)
 - ❑ Puis, tri sur le second («*prenom*»)
 - ❑ ...

Utilisateur		
<u>ID</u>	Nom	Prénom
1	Dupont	Jean
2	Martin	Pierre
3	Nawouak	Bruno
4	Martin	Arthur



<u>ID</u>	Nom	Prénom
1	Dupont	Jean
4	Martin	Arthur
2	Martin	Pierre
3	Nawouak	Bruno



- Ordre croissant par défaut
 - Lexicographique pour les chaînes

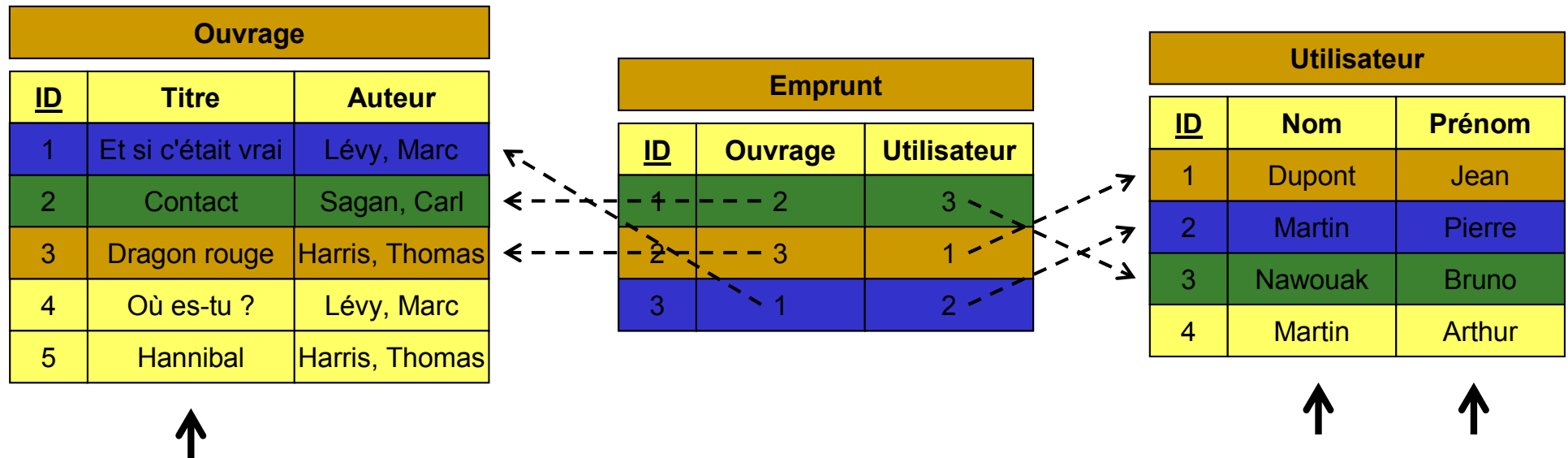
- Décroissant avec le mot-clé «**DESC**»
 - Mot à placer après l'attribut concerné
 - **SELECT * FROM utilisateur**
ORDER BY nom DESC, prenom DESC

- Accélération d'un tri
 - Déclarer l'attribut concerné comme index
 - Mais cela prend plus d'espace



Requêtes SQL: jointure (1/3)

- Fusion de plusieurs tables
- ```
SELECT ouvrage.titre,
 utilisateur.nom,
 utilisateur.prenom
FROM ouvrage, emprunt, utilisateur
WHERE (utilisateur.id=emprunt.utilisateur
 AND ouvrage.id=emprunt.ouvrage)
```





# Requêtes SQL: jointure (2/3)

| Ouvrage   |                    |                |
|-----------|--------------------|----------------|
| <u>ID</u> | Titre              | Auteur         |
| 1         | Et si c'était vrai | Lévy, Marc     |
| 2         | Contact            | Sagan, Carl    |
| 3         | Dragon rouge       | Harris, Thomas |
| 4         | Où es-tu ?         | Lévy, Marc     |
| 5         | Hannibal           | Harris, Thomas |

| Emprunt   |         |             |
|-----------|---------|-------------|
| <u>ID</u> | Ouvrage | Utilisateur |
| 4         | 2       | 3           |
| 2         | 3       | 1           |
| 3         | 1       | 2           |

| Utilisateur |         |        |
|-------------|---------|--------|
| <u>ID</u>   | Nom     | Prénom |
| 1           | Dupont  | Jean   |
| 2           | Martin  | Pierre |
| 3           | Nawouak | Bruno  |
| 4           | Martin  | Arthur |

Jointure

| <u>ID</u> | Titre              | Auteur         | <u>ID</u> | Ouvrage | Utilisateur | <u>ID</u> | Nom     | Prénom |
|-----------|--------------------|----------------|-----------|---------|-------------|-----------|---------|--------|
| 2         | Contact            | Sagan, Carl    | 1         | 2       | 3           | 3         | Nawouak | Bruno  |
| 3         | Dragon rouge       | Harris, Thomas | 2         | 3       | 1           | 1         | Dupont  | Jean   |
| 1         | Et si c'était vrai | Lévy, Marc     | 3         | 1       | 2           | 2         | Martin  | Pierre |



↓ Projection

- Comment distinguer les attributs de même nom ?
  - ❑ Mot-clé «**AS**» pour renommer l'attribut dans la projection
  - ❑ **SELECT \*, ouvrage.id AS ouvrage\_id, emprunt.id AS emprunt\_id FROM ...**



# Requêtes SQL: insertion

---

## ■ Solution 1

- ❑ **INSERT INTO ouvrage (titre, auteur)**  
**VALUES ('Sphère', 'Crichton, Michael')**
- ❑ Les attributs sont précisés
- ❑ Une valeur est affectée à chacun
- ❑ Si un attribut est manquant, une valeur par défaut est attribuée

## ■ Solution 2

- ❑ **INSERT INTO ouvrage VALUES ('', 'Sphère', 'Crichton, Michael')**
- ❑ Les attributs ne sont pas précisés
- ❑ Valeurs affectées selon l'ordre des attributs dans la table
- ❑ Si aucune valeur n'est fournie, une valeur par défaut est attribuée

## ■ Valeur par défaut

- ❑ Soit valeur nulle
  - Chaîne vide ou zéro
- ❑ Soit valeur fixée
  - Indiquée dans la définition de la table
- ❑ Soit valeur entière auto-incrémentée
  - Utilisée pour l'attribution automatique d'une clé primaire



# Requêtes SQL: suppression & mise à jour

---

- Plusieurs enregistrements peuvent être supprimés dans une seule requête !
- **DELETE FROM utilisateur WHERE nom='Nawouak'**
  - Suppression de tous les enregistrements avec le nom «*Nawouak*»
- Pour être sûr de supprimer le bon enregistrement, utiliser sa clé primaire
- **DELETE FROM emprunt WHERE id=2**
  - Suppression de l'enregistrement avec l'identifiant unique «2»
- Idem pour la mise à jour
  - **UPDATE emprunt SET utilisateur=7**
    - Tous les livres sont maintenant empruntés par l'utilisateur «7»
  - **UPDATE emprunt SET utilisateur=7 WHERE ouvrage=2**
    - Le livre «2» est maintenant emprunté par l'utilisateur «7»



# phpMyAdmin: interface d'administration

---

- Interface Web d'administration MySQL
- Niveau administrateur
  - Créer des utilisateurs
    - Login / mot de passe
    - Limitation de l'accès à certaines bases
  - Créer des bases
    - En général, un utilisateur = une base
    - Cas de la plupart des fournisseurs d'espace
- Niveau utilisateur
  - Accès à une ou plusieurs bases
  - Souvent impossible de créer de nouvelles bases
  - Manipulation de tables
    - Définition de leur structure
    - Saisie des données
    - Requêtes



# phpMyAdmin: contenu d'une base (1/2)

Structure de la table

| Structure                                   | SQL                                                                                                                                                                                                                                                                                                                                                                                                                            | Exporter        | Rechercher | Requête           | Opérations | Supprimer  |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|------------|-------------------|------------|------------|
| Table                                       | Actions                                                                                                                                                                                                                                                                                                                                                                                                                        | Enregistrements | Type       | Interclassement   | Taille     | Perte      |
| <input type="checkbox"/> bibliography       |        | 388             | MyISAM     | latin1_swedish_ci | 88,4 Ko    | 364 Octets |
| <input type="checkbox"/> blacklist          |           | 4               | MyISAM     | latin1_swedish_ci | 1,1 Ko     | -          |
| <input type="checkbox"/> coltris_highscore  |           | 343             | MyISAM     | latin1_swedish_ci | 9,2 Ko     | -          |
| <input type="checkbox"/> environment        |           | 1               | MyISAM     | latin1_swedish_ci | 1,0 Ko     | -          |
| <input type="checkbox"/> guestbook          |           | 45              | MyISAM     | latin1_swedish_ci | 12,6 Ko    | 456 Octets |
| <input type="checkbox"/> startris_highscore |           | 266             | MyISAM     | latin1_swedish_ci | 9,0 Ko     | -          |
| <input type="checkbox"/> tetris_access      |           | 31              | MyISAM     | latin1_swedish_ci | 1,7 Ko     | -          |
| <input type="checkbox"/> tetris_highscore   |           | 517             | MyISAM     | latin1_swedish_ci | 14,3 Ko    | -          |
| <input type="checkbox"/> trace              |      | 2               | MyISAM     | latin1_swedish_ci | 1,1 Ko     | -          |
| 9 table(s)                                  | Somme                                                                                                                                                                                                                                                                                                                                                                                                                          | 1 597           | --         | latin1_swedish_ci | 138,3 Ko   | 820 Octets |

Tout cocher / Tout décocher / Cocher tables avec pertes

Pour la sélection :

Version imprimable   Dictionnaire de données

Créer une nouvelle table sur la base nawouak:

Nom:

Champs:

Exécuter

Création d'une table





# phpMyAdmin: structure d'une table

Structure Afficher SQL Rechercher Insérer Exporter Opérations Vider Supprimer

|                          | Champ     | Type     | Interclassement   | Attributs | Null | Défaut | Extra | Action                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------|-----------|----------|-------------------|-----------|------|--------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | code      | tinytext | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | authors   | text     | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | year      | tinytext | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | title     | text     | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | book      | text     | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | publisher | text     | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | volume    | tinytext | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | pages     | tinytext | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | url       | text     | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | category  | tinytext | latin1_swedish_ci |           | Oui  | NULL   |       |                   |
| <input type="checkbox"/> | location  | text     | latin1_swedish_ci |           | Oui  | NULL   |       |       |
| <input type="checkbox"/> | smallcode | tinytext | latin1_swedish_ci |           | Oui  | NULL   |       |       |

Tout cocher / Tout décocher Pour la sélection :      

Version imprimable Suggérer des optimisations quant à la structure de la table

Ajouter 1 champ(s) ☒ En fin de Table ☐ En début de Table ☐ Après code Exécuter

Clé primaire

Structure de l'attribut

Insertion d'un nouvel attribut





# phpMyAdmin: structure d'un attribut

| Champ | Type     | Taille / Valeurs* | Interclassement   | Attributs | Null | Défaut** | Extra |
|-------|----------|-------------------|-------------------|-----------|------|----------|-------|
| code  | TINYTEXT |                   | latin1_swedish_ci |           | null | NULL     |       |

Sauvegarder

\* Les différentes valeurs des champs de type enum/set sont à spécifier sous la forme 'a','b','c'...  
Pour utiliser un caractère "\" ou "'" dans l'une de ces valeurs, faites-le précéder du caractère d'échappement "\" (par exemple '\\xyz' ou 'a\\b').

\*\* Pour les valeurs par défaut, veuillez n'entrer qu'une seule valeur, sans caractère d'échappement ou apostrophes, sous la forme: a

Nom

Domaine

Peut être vide ?

Valeur par défaut

Incrément automatique ?



# phpMyAdmin: contenu d'une base (2/2)

Contenu de la table

| Structure                                   | SQL                                                                                 | Exporter        | Rechercher | Requête           | Opérations | Supprimer  |
|---------------------------------------------|-------------------------------------------------------------------------------------|-----------------|------------|-------------------|------------|------------|
| Table                                       | Action                                                                              | Enregistrements | Type       | Interclassement   | Taille     | Perte      |
| <input type="checkbox"/> bibliography       |    | 388             | MyISAM     | latin1_swedish_ci | 88,4 Ko    | 364 Octets |
| <input type="checkbox"/> blacklist          |    | 4               | MyISAM     | latin1_swedish_ci | 1,1 Ko     | -          |
| <input type="checkbox"/> coltris_highscore  |    | 343             | MyISAM     | latin1_swedish_ci | 9,2 Ko     | -          |
| <input type="checkbox"/> environment        |    | 1               | MyISAM     | latin1_swedish_ci | 1,0 Ko     | -          |
| <input type="checkbox"/> guestbook          |   | 45              | MyISAM     | latin1_swedish_ci | 12,6 Ko    | 456 Octets |
| <input type="checkbox"/> startris_highscore |  | 266             | MyISAM     | latin1_swedish_ci | 9,0 Ko     | -          |
| <input type="checkbox"/> tetris_access      |  | 31              | MyISAM     | latin1_swedish_ci | 1,7 Ko     | -          |
| <input type="checkbox"/> tetris_highscore   |  | 517             | MyISAM     | latin1_swedish_ci | 14,3 Ko    | -          |
| <input type="checkbox"/> trace              |  | 2               | MyISAM     | latin1_swedish_ci | 1,1 Ko     | -          |
| 9 table(s)                                  | Somme                                                                               | 1 597           | --         | latin1_swedish_ci | 138,3 Ko   | 820 Octets |

Tout cocher / Tout décocher / Cocher tables avec pertes

Pour la sélection :



# phpMyAdmin: contenu d'une table

Structure Afficher SQL Rechercher Insérer Exporter Opérations Vider Supprimer

Affichage des enregistrements 0 - 2 (388 total, traitement: 0.0005 sec.)

requête SQL:

```
SELECT *
FROM `bibliography`
LIMIT 0, 3
```

[Modifier] [Expliquer SQL] [Créer source PHP] [Actualiser]

Afficher : 3 ligne(s) à partir de l'enregistrement n° 3  
en mode horizontal et répéter les en-têtes à chaque groupe de 100 Page n°: 1

|                                     |   | code     | authors                                               | year | title                                           | book                                                  | publisher     | volume | pages | url                                     |
|-------------------------------------|---|----------|-------------------------------------------------------|------|-------------------------------------------------|-------------------------------------------------------|---------------|--------|-------|-----------------------------------------|
| <input checked="" type="checkbox"/> | ? | Abello99 | J. Abello, P.M. Pardalos, M.G.C. Resende              | 1999 | On Maximum Clique Problems in Very Large Graphs | External Memory Algorithms, DIMACS Series             |               |        |       | http://citeseer.ist.psu.edu/abello99max |
| <input type="checkbox"/>            | ? | Aho93    | Alfred Aho, Jeffrey Ullman                            | 1993 |                                                 | Concepts fondamentaux de l'informatique               | Dunod         |        |       |                                         |
| <input type="checkbox"/>            | ? | Ahuja93  | Ravindra K. Ahuja, Thomas L. Magnanti, James B. Or... | 1993 |                                                 | Network Flows - Theory, Algorithms, and Applicatio... | Prentice Hall |        |       |                                         |

Tout cocher / Tout décocher Pour la sélection :

Afficher : 3 ligne(s) à partir de l'enregistrement n° 3  
en mode horizontal et répéter les en-têtes à chaque groupe de 100 Page n°: 1

Insérer un nouvel enregistrement Version imprimable Version imprimable (avec textes complets) Exporter

Modification de l'enregistrement

Insertion d'un nouvel enregistrement



# phpMyAdmin: modification d'un enregistrement

| Champ   | Type     | Fonction                                                                                                                                                                                                    | Null                     | Valeur                                          |
|---------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|-------------------------------------------------|
| code    | tinytext | <div>ASCII<br/>CHAR<br/>SOUNDEX<br/>LCASE<br/>UCASE<br/>PASSWORD<br/>MD5<br/>SHA1<br/>ENCRYPT<br/>LAST_INSERT_ID<br/>USER<br/>CONCAT<br/>-----<br/>NOW<br/>RAND<br/>COUNT<br/>AVG<br/>SUM<br/>CURDATE</div> | <input type="checkbox"/> | Abello99                                        |
| authors | text     |                                                                                                                                                                                                             | <input type="checkbox"/> | J. Abello, P.M. Pardalos, M.G.C. Resende        |
| year    | tinytext |                                                                                                                                                                                                             | <input type="checkbox"/> | 1999                                            |
| title   | text     |                                                                                                                                                                                                             | <input type="checkbox"/> | On Maximum Clique Problems in Very Large Graphs |
| book    | text     |                                                                                                                                                                                                             | <input type="checkbox"/> | External Memory Algorithms, DIMACS Series       |

Fonction appliquée  
sur la valeur saisie

(Cryptage du mot de passe par exemple)



# phpMyAdmin: exporter une base

**Formats**

**Exporter**

- ☒ SQL
- ☐ LaTeX
- ☐ CSV pour MS Excel
- ☐ CSV
- ☐ XML

**Schéma et données de la base**

**options SQL ?**

Commentaires mis en en-tête (\n sépare les lignes):

☐ Utiliser le mode transactionnel

☐ Désactiver la vérification des clés étrangères

**Structure:**

- ☐ Inclure des énoncés "DROP TABLE" **Inclut du code pour détruire les tables**
- ☐ Ajouter "IF NOT EXISTS"
- ☒ Inclure la valeur courante de l'AUTO\_INCREMENT **Mémorise les compteurs des clés primaires**
- ☒ Protéger les noms des tables et des champs par des "`" **Utile pour des espaces dans les noms**

**Inclure sous forme de commentaires**

☐ Dates de création/modification/vérification

Compatibilité de l'exportation:

**Données:**

- ☐ Insertions complètes
- ☐ Insertions étendues
- ☐ Insertions avec délais (DELAYED)
- ☐ Ignorer les erreurs de doublons (INSERT IGNORE)
- ☒ Encoder les champs binaires en hexadécimal **Nécessaire dès qu'il y a des données binaires**

Type d'exportation:

**Transmettre**

Modèle de nom de fichier:  ( ☒ se souvenir du modèle )\*

**Compression**

- ☒ aucune
- ☐ "zippé"
- ☐ "gzippé"

**Exécuter**



# phpMyAdmin: importer une base

Exécution de requêtes SQL

The screenshot shows the phpMyAdmin interface with the 'SQL' tab selected. The 'Exécuter une ou des requêtes sur la base nawouak :

Below the text input area, there is a checkbox labeled 'Réafficher la requête après exécution' which is checked, and an 'Exécuter' button.

Under the 'Ou' section, the 'Emplacement du fichier texte:' is set to 'c:\bruno\nawouak.sql', which is circled in green. There is a 'Browse...' button and a note '(Taille maximum: 2 048Ko)'. Below this, the 'Compression:' section has three radio buttons: 'Détection automatique' (selected), 'aucune', and '"gzipé"'. The 'Jeu de caractères du fichier:' is set to 'utf8'.

At the bottom of this section, there is an 'Exécuter' button.

Commandes SQL à importer



# Connexion à un serveur MySQL (1/2)

---

- PHP et MySQL peuvent être sur des serveurs différents
- Informations nécessaires
  - Nom du serveur («*host*»)
    - Si même serveur, *host* = "*localhost*"
  - Identifiant de l'utilisateur («*login*»)
  - Mot de passe («*password*»)
- Informations par défaut dans EasyPHP
  - *login* = "*root*"
  - *password* = ""
  - Problème de sécurité ⇒ changer le mot de passe



## Connexion à un serveur MySQL (2/2)

- Fonction «**mysql\_connect**»
  - Nécessite *host*, *login* et *password*
  - Retourne «**TRUE**» si la connexion est établie
- Si la connexion échoue
  - Un message est affiché dans le navigateur
  - Mode silencieux: **@mysql\_connect**
- Exemple d'utilisation

```
if (@mysql_connect($host,$login,$password)) {
 /* Connexion établie */
}
else die("Connexion au serveur impossible !");
```
- «**@**» empêche l'affichage des erreurs de la fonction qui suit
- «**die**» affiche un message et interrompt le script PHP





# Déconnexion du serveur MySQL

---

- Nombre de connexions souvent limité
  - Pensez aux autres !
  - Se déconnecter le plus tôt possible
  - Penser à se déconnecter même en cas d'erreur
- Fonction «**mysql\_close**»
- Exemple d'utilisation

```
if (@mysql_connect($host,$login,$password)) {
 /* Connexion établie */

 ...
 @mysql_close();
}
```



# Sélection d'une base de données

---

- Fonction «`mysql_select_db`»
  - Nécessite le nom de la base («*base*»)
  - Retourne «**TRUE**» si la sélection a réussi
  
- Exemple d'utilisation

```
if (@mysql_select_db($base)) {
 /* Sélection réussie */
}
else die("Sélection de la base impossible !");
```
  
- Les requêtes se feront sur la base sélectionnée
  - Possibilité de changer de base en cours d'exécution
  - Mais souvent on travaille sur une seule base



# Script complet de connexion (1/3)

---

- Connexion et sélection d'une base de données

```
if (@mysql_connect($host,$login,$password)) {
 if (@mysql_select_db($base)) {
 /* Connexion établie */
 ... // Code du script
 @mysql_close();
 }
 else {
 @mysql_close();
 die("Sélection de la base impossible !");
 }
}
else die("Connexion au serveur impossible !");
```



## Script complet de connexion (2/3)

---

- Script utilisé plusieurs fois
  - A exécuter dans chaque page qui fait une requête
- Ecrire une fonction «**connexion**»
  - Permet d'éviter la recopie dans plusieurs pages
  - Permet d'isoler les identifiants
    - Différents entre site de test et site de production
  - Placer la fonction dans un fichier: **connexion.php**
- Utilisation du fichier  
**`include("connexion.php");`**

```
connexion();
/* Requetes sur la base */
@mysql_close();
```



# Script complet de connexion (3/3)

- Fichier «**connexion.php**»

```
<?php
function connexion() {
 $host = "...";
 $login = "...";
 $password = "...";
 $base = "...";

 if (@mysql_connect($host,$login,$password)) {
 if (@mysql_select_db($base)) return;
 else {
 @mysql_close();
 die("Sélection de la base impossible !");
 }
 }
 else die("Connexion au serveur impossible !");
}
?>
```



# Exécuter une requête SQL

- Requête SQL placée dans une chaîne de caractères
  - `$requete = "SELECT * FROM utilisateur";`
- Exécution de la requête avec «`mysql_query`»
  - `$result = @mysql_query($requete);`
- Retourne le résultat de la requête
  - Si la requête a échoué, retourne «**FALSE**»
  - Sinon «**\$result**» contient le résultat
- Connaître le nombre d'enregistrements
  - Requête «**SELECT**»
    - Nombre d'éléments trouvés  
`$nb = @mysql_num_rows($result);`
  - Requêtes «**DELETE**» et «**UPDATE**»
    - Nombre d'éléments affectés  
`$nb = @mysql_affected_rows();`



# Parcourir le résultat d'une requête (1/4)

---

- Lecture des enregistrements un par un: `mysql_fetch_?`
  - ❑ `$enr = @mysql_fetch_?($result);`
  - ❑ «`$enr`» représente un enregistrement
  - ❑ Cet enregistrement est retiré de «`$result`»
  - ❑ Retourne «`FALSE`» si «`$result`» est vide
  
- Parcours des enregistrements
  - ❑ `while ($enr = @mysql_fetch_?($result)) { ... }`
  
- Trois possibilités pour représenter un enregistrement
  - ❑ Sous forme de tableau simple: `mysql_fetch_row`
  - ❑ Sous forme de tableau associatif: `mysql_fetch_array`
  - ❑ Sous forme d'objet: `mysql_fetch_object`



## Parcourir le résultat d'une requête (2/4)

- Option 1: enregistrement = tableau simple  

```
$requete = "SELECT * FROM utilisateur";
$result = @mysql_query($requete);
```

```
if (!$result) {
 @mysql_close();
 die("La requête a échoué !");
}
```

| Utilisateur |         |        |
|-------------|---------|--------|
| <u>ID</u>   | Nom     | Prénom |
| 1           | Dupont  | Jean   |
| 2           | Martin  | Pierre |
| 3           | Nawouak | Bruno  |

```
while ($enr = @mysql_fetch_row($result)) {
 echo "<p>Utilisateur $enr[0]: $enr[1], $enr[2]</p>";
}
```

- Résultat  

```
<p>Utilisateur 1: Dupont, Jean</p>
<p>Utilisateur 2: Martin, Pierre</p>
<p>Utilisateur 3: Nawouak, Bruno</p>
```





## Parcourir le résultat d'une requête (3/4)

---

- Option 2: enregistrement = tableau associatif

```
while ($enr = @mysql_fetch_array($result)) {
 echo "<p>Utilisateur $enr['id']: ";
 echo "$enr['nom'], $enr['prenom']</p>";
}
```
- Option 3: enregistrement = objet

```
while ($enr = @mysql_fetch_object($result)) {
 echo "<p>Utilisateur $enr->id: ";
 echo "$enr->nom, $enr->prenom</p>";
}
```



# Parcourir le résultat d'une requête (4/4)

---

- Comment choisir une option ?
- Ordre des attributs
  - ❑ Ne peut pas être changé avec l'option 1
  - ❑ N'a aucune importance pour les options 2 et 3
- Noms des attributs
  - ❑ N'ont aucune importance pour l'option 1
  - ❑ Peuvent être changés avec l'option 2
    - A condition d'être stockés dans des variables
  - ❑ Ne peuvent pas être changés avec l'option 3
- Facilité d'écriture
  - ❑ La plus condensée: option 1
  - ❑ La plus longue: option 2
  - ❑ La plus lisible: option 3



# Gérer une requête d'insertion

- Exemple d'une insertion

```
$requete = "INSERT INTO utilisateur
VALUES ('', 'Martin', 'Arthur')";
$result = @mysql_query($requete);
```

Utilisateur		
<u>ID</u>	Nom	Prénom
1	Dupont	Jean
2	Martin	Pierre
3	Nawouak	Bruno

Insertion →

Utilisateur		
<u>ID</u>	Nom	Prénom
1	Dupont	Jean
2	Martin	Pierre
3	Nawouak	Bruno
4	Martin	Arthur

- L'index du nouvel enregistrement peut être utile
- Mais comment le récupérer si il est incrémenté automatiquement ?
  - ❑ `$id = @mysql_insert_id();`
  - ❑ Retourne la valeur de l'attribut en auto-incrément



# PARTIE IV

## Gestion d'une base de données par interface Web

**Copyright (c) 1999-2011 - Bruno Bachelet - [bruno@nawouak.net](mailto:bruno@nawouak.net) - <http://www.nawouak.net>**

La permission est accordée de copier, distribuer et/ou modifier ce document sous les termes de la licence *GNU Free Documentation License*, Version 1.1 ou toute version ultérieure publiée par la fondation *Free Software Foundation*. Voir cette licence pour plus de détails (<http://www.gnu.org>).



# Paramètres d'une page PHP

---

- Une page PHP peut recevoir des paramètres
- D'un formulaire
  - ❑ Données à ajouter dans la base
  - ❑ Données à rechercher dans la base
  - ❑ ...
- D'un lien hypertexte
  - ❑ Paramètre d'affichage (e.g. critère de tri)
  - ❑ Identifiant de l'élément à afficher (e.g. ID d'un ouvrage)
  - ❑ ...
- Méthode «*get*»
  - ❑ Données en clair dans l'URL
- Méthode «*post*»
  - ❑ Données non visibles dans l'URL



# Paramètres avec la méthode «get» (1/2)

---

- Paramètres transmis dans l'URL
  - Exemple: `liste.php?tri=auteur`
- Sont lisibles dans le navigateur
- La taille des données est limitée
- Attention au format des données
  - Problème avec les caractères spéciaux
- Paramètres dans le tableau associatif «`_GET`»
  - Clé = nom du paramètre
  - Élément = valeur du paramètre
  - `$colonne = $_GET["tri"];`



## Paramètres avec la méthode «get» (2/2)

- Page «liste.php» avec paramètre

```
<html>
<body>
 <?php
 include("connexion.php");
 $colonne = $_GET["tri"];

 connexion();
 $requete = "SELECT * FROM ouvrage ORDER BY $colonne";
 $result = @mysql_query($requete);

 while ($enr = @mysql_fetch_object($result)) {
 echo "<p>$enr->auteur - $enr->titre</p>";
 }

 @mysql_close();
 ?>
</body>
</html>
```

- Exemple d'utilisation

```
<p>Tri par auteur</p>
<p>Tri par titre</p>
```



# HTML et les formulaires

---

- Formulaire = balise HTML **<form>**
- Contient des champs de saisie
  - ❑ Chaque champ a un nom (attribut «**name**»)
  - ❑ Chaque champ a une valeur
- Résultat du formulaire = association nom-valeur
  - ❑ *nom1 = valeur1 (nom = Nawouak)*
  - ❑ *nom2 = valeur2 (prenom = Bruno)*
  - ❑ ...
- Un formulaire est associé à une action
  - ❑ Action = URL où transmettre les données
  - ❑ Un bouton de soumission déclenche l'action
- Données transmises = association nom-valeur
  - ❑ *paramètre1 = valeur1 (nom = Nawouak)*
  - ❑ *paramètre2 = valeur2 (prenom = Bruno)*





# Exemple de formulaire (1/2)

- Code HTML

```
<form action="login.php">
 <p>Utilisateur:
 <input type="text" name="user"/></p>
 <p>Mot de passe:
 <input type="password" name="pass"/></p>
 <p><input type="submit" value="Se connecter"/></p>
</form>
```

- Apparence

Utilisateur:

Mot de passe:

- Résultat

*user = Nawouak*

*pass = 12345*



## Exemple de formulaire (2/2)

- Soumission du formulaire
  - ❑ Transmission des paramètres à «**login.php**»
  - ❑ Redirection vers: **login.php?user=Nawouak&pass=12345**
  - ❑ «**?**» démarre la séquence des paramètres
  - ❑ «**&**» sépare deux paramètres
  
- Récupération des paramètres dans «**login.php**»  
**\$utilisateur = \$\_GET["user"];**  
**\$mdp = \$\_GET["pass"];**
  
- Paramètre n'existe pas  $\Rightarrow$  message d'erreur
  - ❑ «**@**» permet de masquer l'erreur
    - **\$utilisateur = @\$\_GET["user"];**
    - En cas d'erreur: **\$utilisateur = "";**
  - ❑ Autre possibilité: tester l'existence du paramètre
    - **if (isset(\$\_GET["user"])) { /\* "user" existe \*/ }**



# Formulaire: saisie de texte (1/2)

---

- Balise `<input type="text"/>`

- ❑ Saisie d'une seule ligne
- ❑ Texte visible

- Valeur par défaut

- ❑ Attribut «**value**»

- Exemple

`<input type="text" name="user" value="votre nom ?"/>`

- Balise `<input type="password"/>`

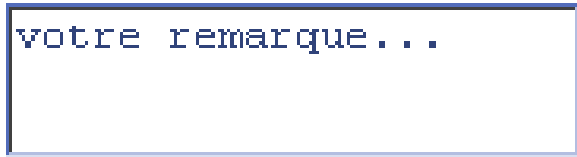
- ❑ Idem «**text**» mais texte masqué à la saisie



## Formulaire: saisie de texte (2/2)

- Balise `<textarea>`
  - Saisie sur plusieurs lignes
- Valeur par défaut
  - Placée dans la balise
- Taille de la zone de texte
  - Attribut «`rows`»: hauteur, en nombre de lignes
  - Attribut «`cols`»: largeur, en nombre de caractères
- Exemple

```
<textarea name="remarque" rows="3" cols="22">
 votre remarque...
</textarea>
```



votre remarque...



# Formulaire: sélection (1/3)

- Balise `<input type="checkbox"/>`
  - Réponse oui / non
- Valeur par défaut
  - Attribut «**checked**»
  - Pour cocher: **checked="yes"**
- Transmission
  - Coché ⇒ paramètre avec valeur «*on*» ou valeur de l'attribut «**value**»
  - Non coché ⇒ paramètre non transmis
- Exemple

```
<p><input type="checkbox" value="ok"
 name="student"/>Etudiant</p>
<p><input type="checkbox" checked="yes"
 name="worker"/>Travailleur</p>
```

☐ Etudiant  
☒ Travailleur



## Formulaire: sélection (2/3)

- Balise `<input type="radio"/>`
  - Permettre une sélection parmi plusieurs choix
- Fonctionnement similaire à «checkbox»
- Un seul bouton doit être sélectionné
  - Les boutons sont liés par le même nom (attribut «name» identique)
- Bouton sélectionné par défaut
  - Celui avec: `checked="yes"`
- Valeur transmise = valeur du bouton sélectionné

- Exemple

```
<p><input type="radio" name="freq"
value="jour"/>tous les jours</p>
<p><input type="radio" name="freq" value="sem"
checked="yes"/>toutes les semaines</p>
<p><input type="radio" name="freq"
value="mois"/>tous les mois</p>
```

- ☐ tous les jours
- ☒ toutes les semaines
- ☐ tous les mois



## Formulaire: sélection (3/3)

- Balise `<select>`
  - Menu déroulant
- Une option du menu = une balise `<option>`
  - `<option value="valeur">intitulé</option>`
- Valeur transmise = valeur de l'option sélectionnée

- Exemple

```
<select name="freq">
 <option value="jour">
 tous les jours</option>
 <option value="sem" selected="selected">
 toutes les semaines</option>
 <option value="mois">
 tous les mois</option>
</select>
```



# Formulaire: champ caché

---

- Il peut être nécessaire de transmettre des données qui ne sont pas saisies par l'utilisateur
- Exemple: l'identifiant de l'utilisateur
  - Pour stocker en base de données l'auteur d'une saisie
- Balise `<input type="hidden"/>`
- Valeur transmise = valeur dans l'attribut «**value**»
- Exemple

```
<input type="hidden" name="user"
value=<?php echo "\"$user_id\""; ?> />
```





# Formulaire: boutons

- Balise `<input type="submit"/>`
  - ❑ Clic  $\Rightarrow$  soumission du formulaire
  - ❑ Attribut «**value**» change le texte du bouton
- Balise `<input type="reset"/>`
  - ❑ Clic  $\Rightarrow$  réinitialisation du formulaire
  - ❑ Attribut «**value**» change le texte du bouton
- Balise `<input type="image"/>`
  - ❑ Bouton remplacé par une image
  - ❑ Attribut «**src**» contient l'image du bouton
  - ❑ Clic  $\Rightarrow$  soumission du formulaire
- Si un bouton possède un nom
  - ❑ Au clic, le nom est envoyé comme paramètre avec sa valeur
- Exemple
  - ❑ `<input type="submit" value="Soumettre" name="action"/>`
  - ❑ Au clic, transmission du paramètre: *action = Soumettre*





## Paramètres avec la méthode «*post*» (1/3)

---

- Paramètres transmis dans l'entête de la requête
  - ❑ Provient d'un formulaire
  - ❑ Exemple: champ «**user**» avec la valeur «**Nawouak**»
- Ils n'apparaissent pas en clair dans l'URL
  - ❑ Mais ne sont pas pour autant cryptés
- La taille des données n'est plus limitée
- Il n'y a pas à se soucier des caractères spéciaux
- Paramètres dans le tableau associatif «**\_POST**»
  - ❑ Clé = nom du paramètre
  - ❑ Élément = valeur du paramètre
  - ❑ **\$utilisateur = \$\_POST["user"];**



## Paramètres avec la méthode «*post*» (2/3)

---

- Préciser la méthode de transmission
  - Attribut «**method**» du formulaire
  - Valeurs: «*get*» ou «*post*»
  - Par défaut: «*get*»
- Exemple

```
<form method="post" action="ajout.php">
 <p>Nom:
 <input type="text" name="lastname"/></p>
 <p>Prénom:
 <input type="text" name="firstname"/></p>
 <p><input type="submit"/></p>
</form>
```
- Transmission à la page «**ajout.php**»
  - Sans paramètres visibles !



# Paramètres avec la méthode «*post*» (3/3)

- Page «**ajout.php**» avec paramètres

```
<?php
include("connexion.php");
$nom = @$_POST["lastname"];
$prenom = @$_POST["firstname"];

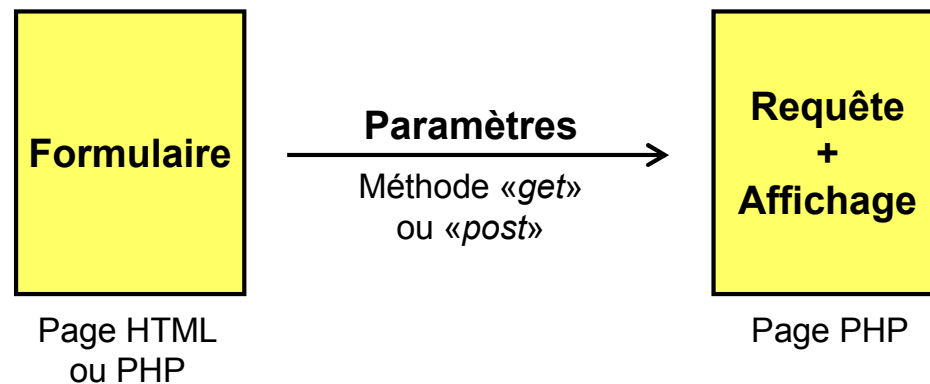
connexion();
$requete = "INSERT INTO utilisateur
 VALUES ('', '$nom', '$prenom')";
@mysql_query($result);
@mysql_close();
?>
```

- Remarque: la page est invisible
  - ❑ Elle ajoute un enregistrement dans la base
  - ❑ Ensuite elle doit rediriger sur une autre page
- Redirection sur une autre page
  - ❑ **header("Location: nouvelle\_page");**
  - ❑ Attention: rien ne doit être affiché avant cette instruction
  - ❑ Doit être avant toute balise HTML éventuelle
  - ❑ Pas d'instruction «**echo**» avant



# Schémas classiques de transmission (1/6)

- Consultation de la base de données
  - Page formulaire
  - Page requête (avec affichage HTML)
- Transmission par les méthodes «*get*» ou «*post*»
  - «*get*» seulement s'il y a besoin d'un lien hypertexte





# Schémas classiques de transmission (2/6)

---

- Formulaire

```
<html>
 <body>
 <form method="post" action="search.php">
 <p>Mot:
 <input type="text" name="mot"/></p>
 <p><input type="submit" value="Chercher"/></p>
 </form>
 </body>
</html>
```

- Transmission

- *mot* = ?



# Schémas classiques de transmission (3/6)

---

- Requête + affichage

```
<html>
<body>
 <?php
 include("connexion.php");
 $mot = @$_POST["mot"];

 connexion();
 $requete = "SELECT * FROM ouvrage";
 $requete .= " WHERE titre LIKE '%$mot%'";
 $result = @mysql_query($requete);

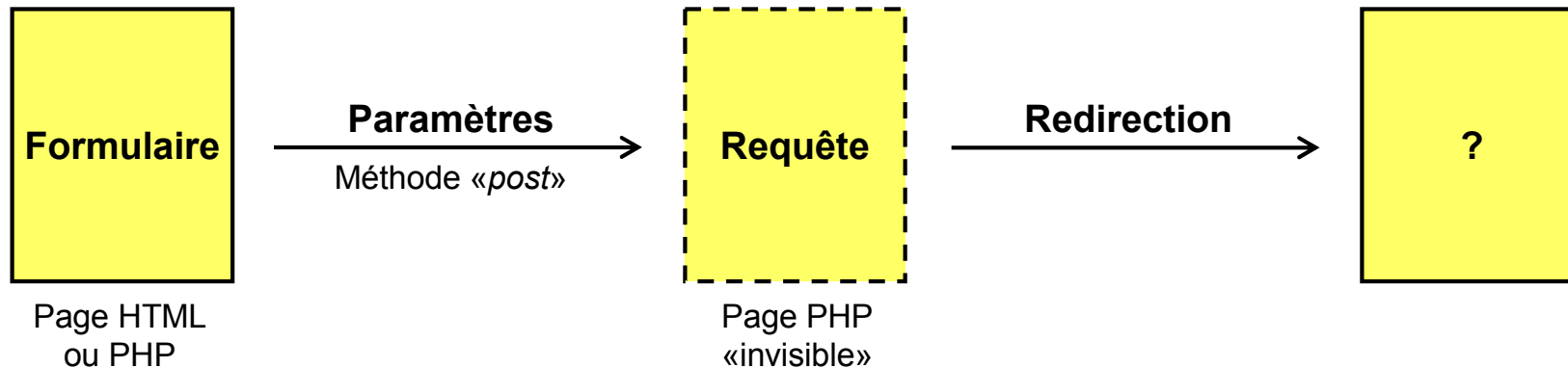
 while ($enr = @mysql_fetch_object($result)) {
 echo "<p>$enr->titre ($enr->auteur)</p>";
 }

 @mysql_close();
 ?>
</body>
</html>
```



# Schémas classiques de transmission (4/6)

- Modification de la base de données
  - Page formulaire
  - Page requête «invisible»
    - Aucun affichage
    - Redirection sur une autre page après la requête
- Transmission par la méthode «*post*»







# Schémas classiques de transmission (5/6)

---

- Formulaire

```
<html>
 <body>
 <form method="post" action="add_livre.php">
 <p>Auteur:
 <input type="text" name="auteur"/></p>
 <p>Titre:
 <input type="text" name="titre"/></p>
 <p><input type="submit" value="Ajouter"/></p>
 </form>
 </body>
</html>
```

- Transmission

- ❑ *auteur* = ?
- ❑ *titre* = ?



# Schémas classiques de transmission (6/6)

- Requête sans affichage

```
<?php
 include("connexion.php");
 $auteur = addslashes(@$_POST["auteur"]);
 $titre = addslashes(@$_POST["titre"]);

 connexion();
 $requete = "INSERT INTO ouvrage
 VALUES ('', '$auteur', '$titre')";
 @mysql_query($requete);
 @mysql_close();

 header("Location: home.htm");
?>
```

- Pour rester invisible (et permettre la redirection)

- ❑ Pas de code HTML dans cette page
- ❑ Pas de commande «**echo**»

- Attention à la saisie de texte

- ❑ Possibilité de saisir du texte avec les symboles «'» et «"»
- ❑ Pour écrire la requête SQL, il faut les remplacer par «\'» et «\"»
- ❑ Pour cela, on utilise la fonction «**addslashes**»



# PARTIE V

## Authentification des utilisateurs d'un site Internet

**Copyright (c) 1999-2011 - Bruno Bachelet - [bruno@nawouak.net](mailto:bruno@nawouak.net) - <http://www.nawouak.net>**

La permission est accordée de copier, distribuer et/ou modifier ce document sous les termes de la licence *GNU Free Documentation License*, Version 1.1 ou toute version ultérieure publiée par la fondation *Free Software Foundation*. Voir cette licence pour plus de détails (<http://www.gnu.org>).



# Mémoriser des données entre pages

---

- Exemple: l'identifiant d'un utilisateur
  - Vérification de l'identité d'un utilisateur
  - Ensuite, utilisation de son identifiant
  
- Méthodes de transfert «*get*» et «*post*»
  - Transfert de données entre pages
  - Echange de l'identifiant de l'utilisateur à chaque fois
    - Lourdeur du procédé
    - Problème de sécurité
  
- Idéalement, il faudrait des «variables globales»
  - Zone d'échange entre différentes pages
  - Il existe deux mécanismes
    - Côté client: les cookies
    - Côté serveur: les sessions

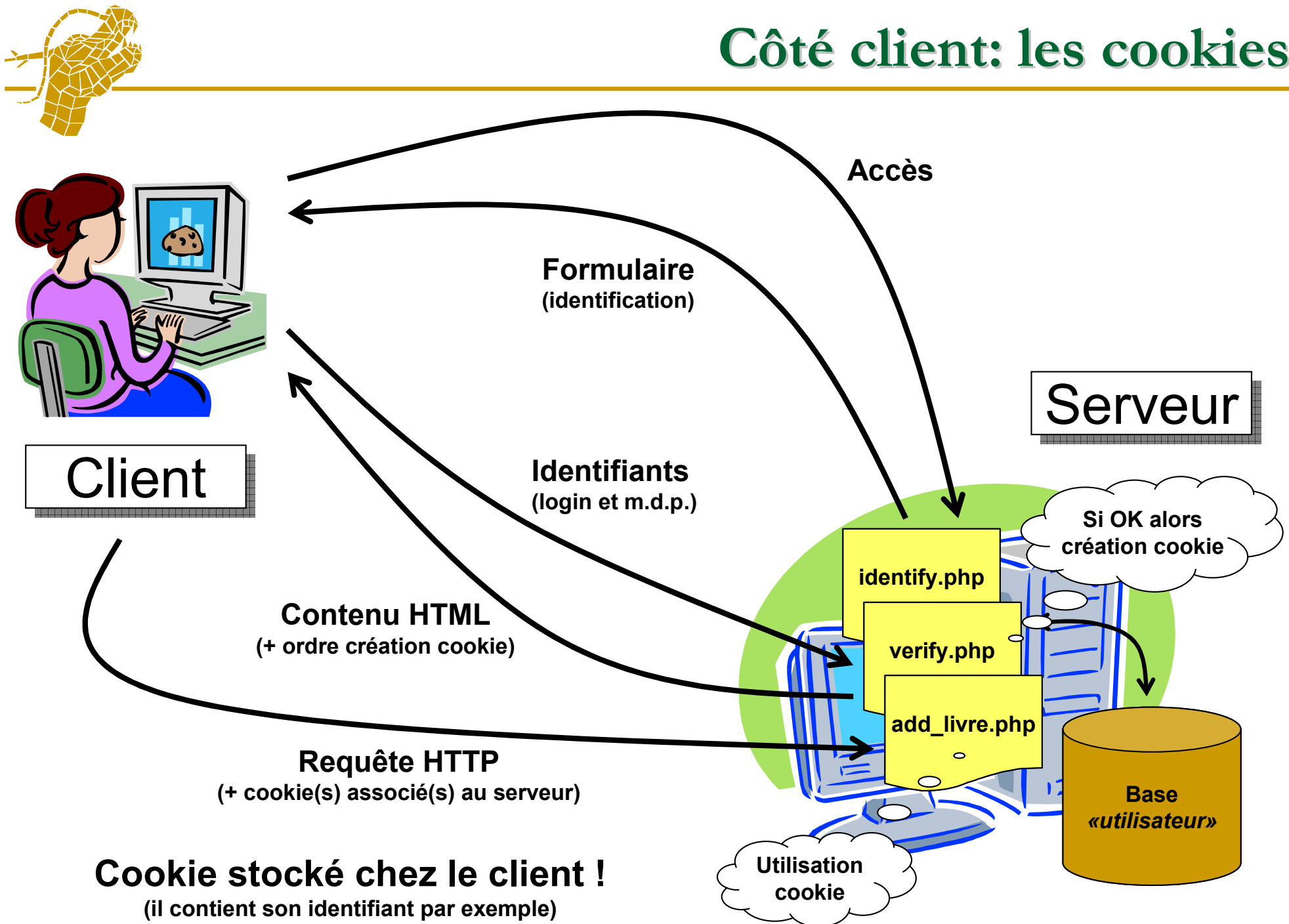


# Qu'est-ce qu'un cookie ?

---

- Donnée stockée par un serveur chez le client
  - ❑ Ordre du serveur en réponse à une requête HTTP
  - ❑ Reçu et interprété par le navigateur
  - ❑ Données stockées sous forme texte sur la machine
  
- Mémoire de l'information entre deux visites
  - ❑ A chaque requête HTTP, le(s) cookie(s) sont transmis
  - ❑ Seuls les cookies du site concerné sont envoyés
  
- Permet de mémoriser
  - ❑ Identifiant d'un utilisateur
  - ❑ Préférences / options sur le site
  - ❑ Pages visitées par un client

# Côté client: les cookies





# Avantages et inconvénients des cookies

---

## ■ Avantages

- ❑ Peut conserver des données entre deux visites
- ❑ Durée de vie des données contrôlée
  - Mort programmée par le serveur
- ❑ Le client peut détruire les données
  - Nettoyage par le navigateur

## ■ Inconvénients

- ❑ Le client peut refuser les cookies
- ❑ Quantité de données stockées limitée
- ❑ Problème de sécurité
  - Echange de données entre client et serveur
    - ❑ Par défaut, aucune sécurisation
    - ❑ Possibilité de sécuriser le transfert
  - Données stockées chez le client
    - ❑ Possibilité de récupérer le cookie d'un autre



# Qu'est-ce qu'une session ?

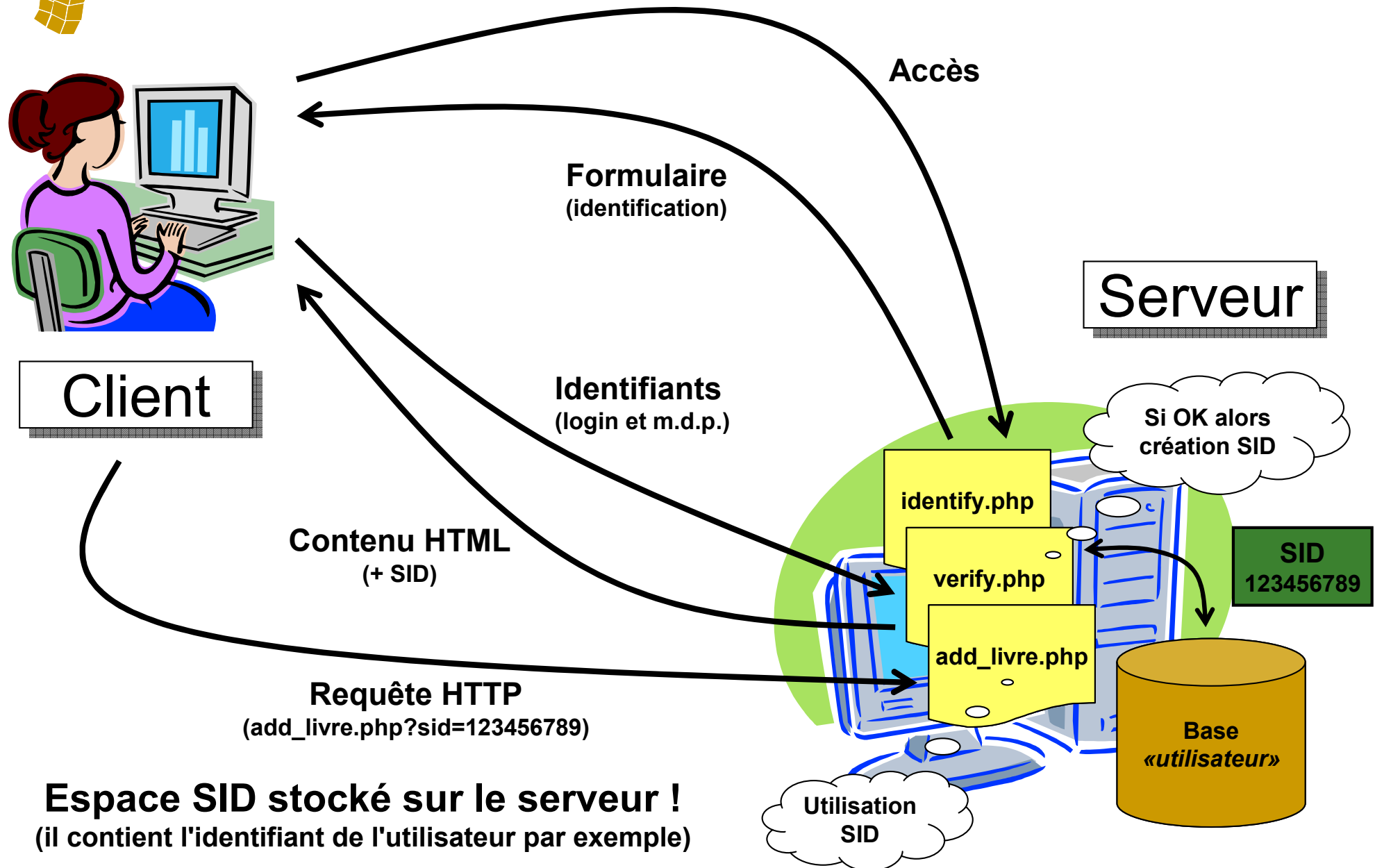
---

- Correspond à une visite d'un utilisateur
  - ❑ Identifiant attribué aléatoirement au début de la visite
  - ❑ Identifiant transmis à chaque page durant la visite
  - ❑ Identifiant = SID (*Session IDentifier*)
- L'identifiant permet d'échanger des données entre pages
  - ❑ Zone de stockage attribuée pour l'identifiant sur le serveur
  - ❑ Une page peut déposer des informations dans la zone
  - ❑ Une autre peut récupérer ces informations dans la zone
- L'identifiant doit être transmis entre les pages
  - ❑ Plusieurs méthodes de transmission





# Côté serveur: les sessions





- Fonction «**session\_start**» ouvre une session
  - Création d'un nouvel SID si l'utilisateur n'en a pas
  - Sinon, récupération du SID de l'utilisateur
  - A partir du SID, récupération des données de la session
  - Procédure totalement automatique
- Données stockées dans le tableau associatif «**\_SESSION**»
  - Permet de sauver des informations
    - Association variable-valeur
    - **\$\_SESSION["nom\_variable"] = valeur;**
  - Permet de récupérer des informations déjà stockées
    - **echo \$\_SESSION["nom\_variable"];**
- Suppression d'une session ⇔ «*logout*»
  - **session\_destroy();**
  - Données de la session détruites
  - SID rendu invalide
- Utiliser ces commandes avant tout affichage
  - Avant la balise **<html>**
  - Avant toute commande «**echo**»



# Avantages et inconvénients des sessions

---

## ■ Avantages

- ❑ Fonctionne toujours, le client ne peut pas refuser
- ❑ Aucune limite à la quantité de données stockées
- ❑ Pas d'échange de données avec le client
  - Plus de problème de sécurité
  - Données uniquement stockées sur le serveur et consultables seulement à partir du SID

## ■ Inconvénients

- ❑ Durée de vie limitée au temps de la visite
  - Impossible de conserver des données entre deux visites
- ❑ La session peut être fermée pendant la visite
  - Durée de vie de la session programmée
  - Réinitialisation du compteur de vie à chaque accès au site
  - Aucun accès pendant une durée donnée  
⇒ fermeture de la session



# Authentifier un utilisateur (1/4)

---

- Page de connexion
  - ❑ Saisie du login et du mot de passe
  - ❑ Vérification de leur validité
  - ❑ Sauvegarde login + m.d.p. dans une session
  
- Autres pages
  - ❑ Récupération login + m.d.p. de la session
  - ❑ Vérification de leur validité
  - ❑ Si non valide, retour à la page de connexion
  
- Pourquoi stocker login + m.d.p. au lieu de l'identifiant ?
  - ❑ L'identifiant peut devenir invalide
  - ❑ Exemple: utilisateur supprimé pendant la visite



## Authentifier un utilisateur (2/4)

---

- On suppose une table «*utilisateur*»
  - «*id*»: clé primaire, numéro de l'enregistrement
  - «*login*»: chaîne de caractères, identifiant de l'utilisateur
  - «*mdp*»: chaîne de caractères, mot de passe de l'utilisateur
- Formulaire pour saisir login + m.d.p.

```
<form method="post" action="login.php">
 <p>Identifiant:
 <input type="text" name="login"/></p>
 <p>Mot de passe:
 <input type="password" name="pass"/></p>
 <p><input type="submit"/></p>
</form>
```
- Transmission au fichier «**login.php**»



# Authentifier un utilisateur (3/4)

- Vérification du mot de passe: page «**login.php**»

```
<?php
include("connexion.php");
$login = @$_POST["login"];
$mdp = @$_POST["pass"];

connexion();
$requete = "SELECT * FROM utilisateur
 WHERE login='$login' AND mdp='$mdp'";
$result = @mysql_query($requete);
$nb_ligne = @mysql_num_rows($result);
@mysql_close();

if ($nb_ligne == 0) {
 header("Location: connexion_prob.php");
 return;
}

/* Utilisateur authentifié */
session_start();
$_SESSION["login"]=$login;
$_SESSION["pass"]=$mdp;
header("Location: connexion_ok.php");
?>
```



# Authentifier un utilisateur (4/4)

- Dans chaque page, vérifier les identifiants

```
<?php
include("connexion.php");
session_start();
$login = @$_SESSION["login"];
$mdp = @$_SESSION["pass"];

connexion();
$requete = "SELECT * FROM utilisateur
 WHERE login='$login' AND mdp='$mdp'";
$result = @mysql_query($requete);
$nb_ligne = @mysql_num_rows($result);
@mysql_close();

if ($nb_ligne == 0) {
 header("Location: connexion_prob.php");
 return;
}

/* Utilisateur authentifié */
...
?>
```



# Sécurité: pourquoi crypter les mots de passe ?

---

- Le problème
  - ❑ Souvent, même mot de passe pour plusieurs accès
  - ❑ Les administrateurs peuvent voir les mots de passe
  
- Le risque
  - ❑ Serveur craqué = personne non autorisée devient administrateur
  - ❑ Les mots de passe permettent l'accès à des sites plus sensibles
  
- Cryptage par fonction de hachage
  - ❑ Considéré comme une forme puissante de cryptage
  - ❑ Génère une valeur quasi unique à partir d'un texte
  - ❑ Processus supposé à sens unique
  - ❑ Puissance de calcul inimaginable pour inverser le processus





# Cryptage des mots de passe (1/2)

Champ	Type	Fonction	Null	Valeur
id	int(11)			
login	text			Nawouak
mdp	text			dummy
valide	char(1)			0
nom	text			Bachelet
prenom	text			Bruno
email	text			bruno@nawouak.net

Mot de passe saisi en clair

Fonction de cryptage

**C'est le mot de passe crypté  
qui sera stocké en base !**

**Impossible de retrouver  
le mot de passe original**



## Cryptage des mots de passe (2/2)

- Utilisation de la fonction de cryptage de MySQL
  - Vérification

```
SELECT * FROM utilisateur
WHERE (login='$login' AND mdp=MD5('$password'))
```
  - Ajout

```
INSERT INTO utilisateur (login,mdp)
VALUES ('$login',MD5('$password'))
```
- Utilisation de la fonction de cryptage de PHP
  - `if ($enr["mdp"] == md5($password)) ...`
- Attention: l'encodage de l'attribut est précis !

	Champ	Type	Interclassement	Attributs	Null	Défaut	Extra	Action					
<input type="checkbox"/>	id	int(11)			Non		auto_increment						
<input type="checkbox"/>	login	text	latin1_swedish_ci		Oui	NULL							
<input type="checkbox"/>	mdp	text	utf8_general_ci		Oui	NULL							
<input type="checkbox"/>	valide	char(1)	latin1_swedish_ci		Oui	0							
<input type="checkbox"/>	nom	text	latin1_swedish_ci		Oui	NULL							
<input type="checkbox"/>	prenom	text	latin1_swedish_ci		Oui	NULL							
<input type="checkbox"/>	email	text	latin1_swedish_ci		Oui	NULL							



# PARTIE VI

## Modélisation d'un site Internet

**Copyright (c) 1999-2011 - Bruno Bachelet - [bruno@nawouak.net](mailto:bruno@nawouak.net) - <http://www.nawouak.net>**

La permission est accordée de copier, distribuer et/ou modifier ce document sous les termes de la licence *GNU Free Documentation License*, Version 1.1 ou toute version ultérieure publiée par la fondation *Free Software Foundation*. Voir cette licence pour plus de détails (<http://www.gnu.org>).



# Qu'est-ce que la modélisation ?

---

- Un problème est posé
  - Problème = système + question
  
- Types de systèmes
  - Organisationnels: entreprise, école...
  - Biologiques: organisme, écosystème...
  - Informatiques: réseaux, logiciel, système d'information...
  - ...
  
- Système d'information
  - Base de données
  - Interface homme-machine (IHM) permettant la gestion
  
- Questions possibles
  - Proposer un système répondant aux besoins
  - Les besoins changent  $\Rightarrow$  modification du système
  - Optimisation du système
  - ...



# Etapes de la modélisation

---

- Question  $\Rightarrow$  réponse
- Pour répondre, il faut un ou plusieurs modèles
- Modèle = représentation simplifiée d'un système
  - Plusieurs modèles possibles pour un système
  - Ne sont représentés que les éléments pertinents pour l'étude
- Trois grandes étapes pour répondre
  - Analyse  $\Rightarrow$  1<sup>er</sup> modèle
    - Déterminer les besoins
    - Etudier le système existant éventuel
  - Conception  $\Rightarrow$  2<sup>ème</sup> modèle
    - Apporter une réponse théorique à la question
  - Implémentation  $\Rightarrow$  système réel
    - Réaliser la solution (exemple: programme informatique)



- Objectif: formaliser le problème
- Effectuer un état des lieux
  - De l'existant
  - Des souhaits du commanditaire
- Pour un site Web
  - Etudier le site existant
  - Etudier le cahier des charges
- Résultat: un modèle représentant la demande



- Objectif: apporter une solution au problème
- Solution «conceptuelle»
  - ❑ Réfléchir à une solution complète
  - ❑ En se passant des détails techniques
  - ❑ Solution indépendante des langages et outils informatiques
- Pour un site Web
  - ❑ Proposer une base de données
  - ❑ Proposer une interface Web
- Résultat: un modèle représentant une solution



## 3<sup>ème</sup> étape: implémentation

---

- Objectif: réaliser la solution proposée
- Solution «technique»
  - ❑ Les détails techniques sont abordés
  - ❑ Réaliser la solution conceptuelle proposée
  - ❑ Choix des outils et langages informatiques
- Pour un site Web
  - ❑ Choisir le ou les langages de programmation
  - ❑ Créer les bases de données
  - ❑ Ecrire les pages HTML et scripts
- Résultat: une solution fonctionnelle





# Modélisation d'un site Web

---

- Phase d'analyse
  - Besoins + existant
- Phase de conception
  - Base de données + interface du site
- Trois grandes étapes de modélisation d'un site
  - Evaluer les besoins et l'existant
    - Diagramme des cas d'utilisation (UML)
    - Diagramme de classes (UML)
  - Définir la structure de la base de données
    - Modèle de données entité-association
  - Définir l'interface du site
    - Schéma de navigation (relations entre pages)
    - Esquisse des pages (structure des pages)



- UML = *Unified Modeling Language*
- Formalisme international pour la modélisation
  - Permet de modéliser tout type de système
  - Utilisé en particulier pour modéliser des logiciels
    - Programmes informatiques
    - Bases de données
    - Sites Web
- Principaux types de schémas
  - Diagramme des cas d'utilisation
    - Analyse des besoins
  - Diagramme de classes
    - Structure statique du système
  - Diagramme d'interactions
    - Structure dynamique du système



# 1<sup>ère</sup> étape: les besoins et l'existant

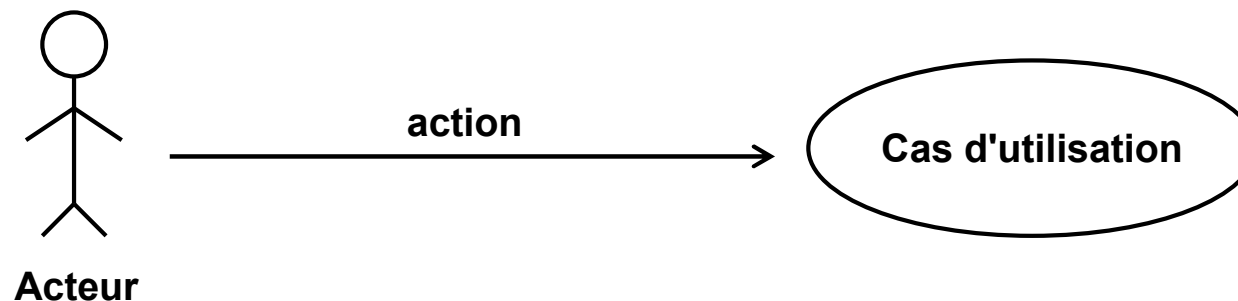
---

- Identifier les scénarios d'utilisation du système
  - Les acteurs
    - Leur rôle dans le système
    - Les relations entre acteurs
    - Hiérarchie des rôles ?
  - Leurs actions
    - Consultation du contenu
    - Contribution au contenu
    - Gestion du site et des utilisateurs
- Pour en déduire la structure du système
- Modélisation
  - 1<sup>er</sup> modèle: les scénarios d'utilisation
  - 2<sup>nd</sup> modèle: la structure du système



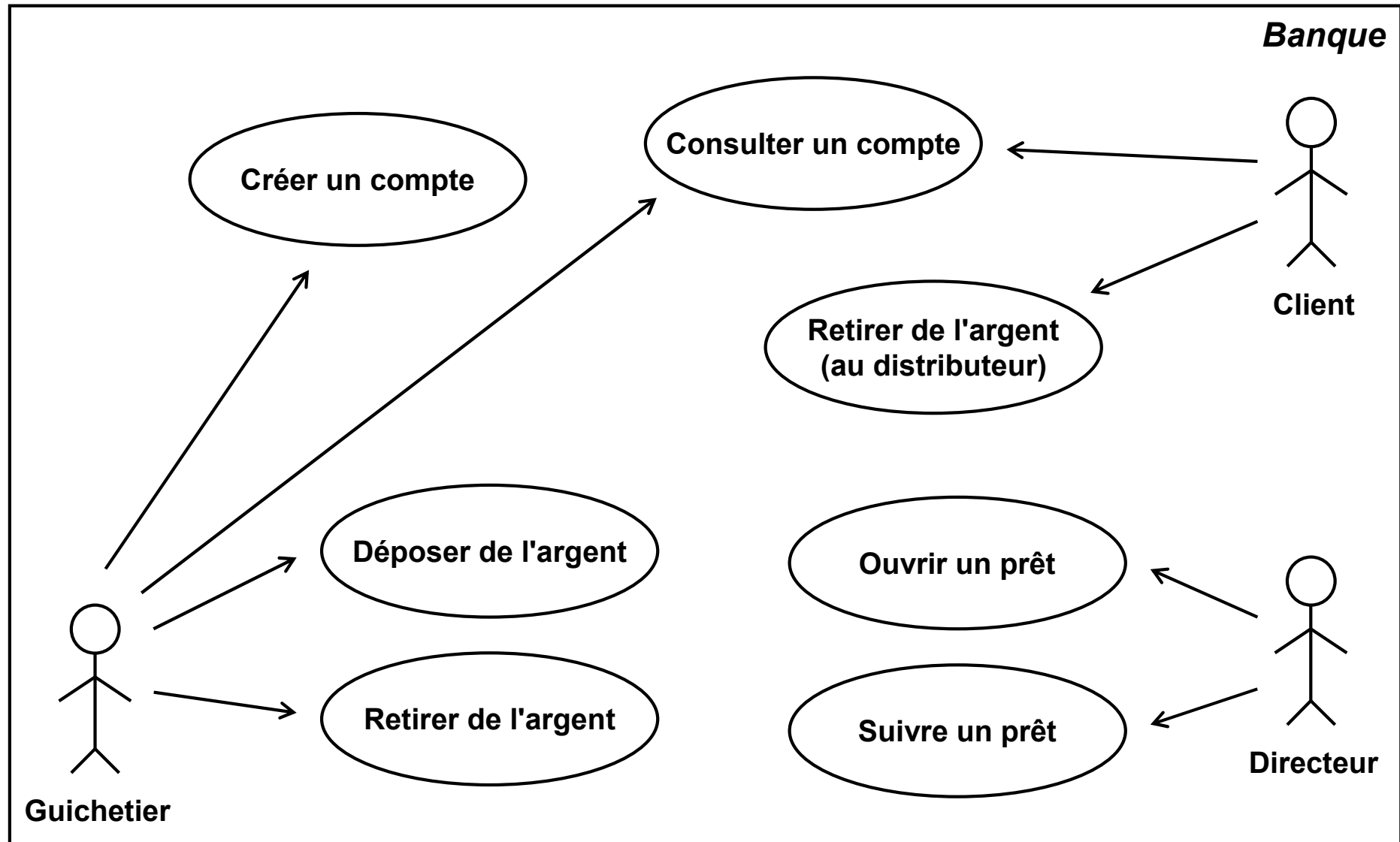
# Diagramme des cas d'utilisation (1/3)

- Décrire le système d'un point de vue utilisateur
- Cas d'utilisation = scénario d'utilisation
  - ❑ Manière spécifique d'utiliser le système
  - ❑ Fonctionnalité déclenchée sur le système
- Acteur
  - ❑ Entité externe qui agit sur le système
  - ❑ Représente un rôle par rapport au système
- Cas d'utilisation
  - ❑ Déclenché par un acteur (suite à une action)
  - ❑ Ensemble d'actions sur le système
  - ❑ Fonction visible par l'acteur





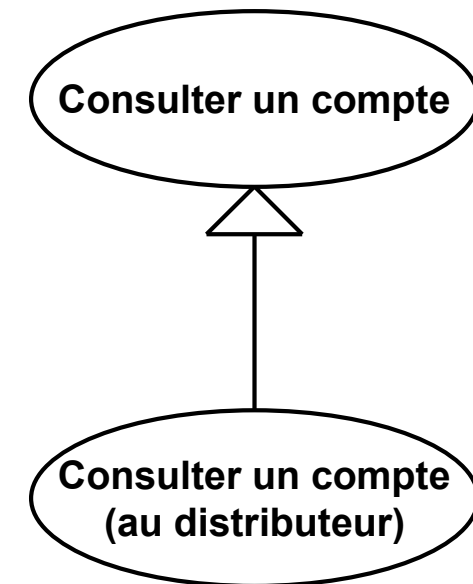
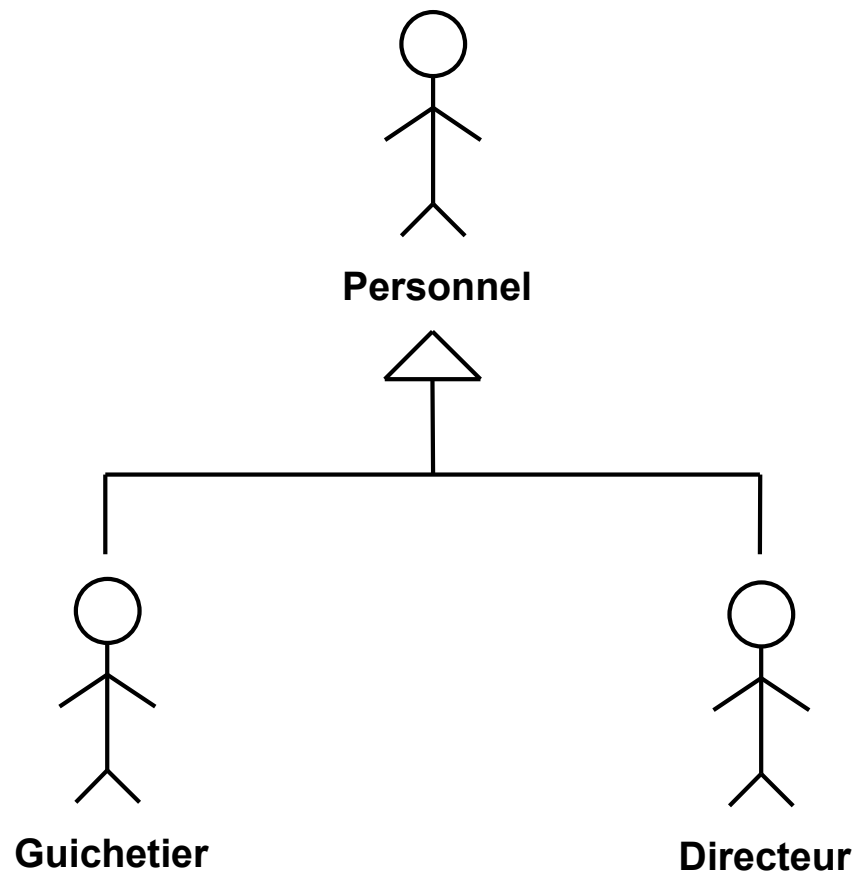
# Diagramme des cas d'utilisation (2/3)





# Diagramme des cas d'utilisation (3/3)

- Héritage entre acteurs (respectivement cas d'utilisation)
  - Héritage du rôle (respectivement des actions)
  - Enrichissement du rôle (respectivement des actions)





# Diagramme de classes (1/5)

- Permet de représenter la structure du système
- Décrit des relations entre types d'entités
- Une classe regroupe des entités (objets) ayant
  - Des propriétés (des attributs) similaires
  - Un comportement (des méthodes) commun
  - Des relations communes avec d'autres objets

## Formalisme

Nom classe
+ attribut 1: type + attribut 2: type ...
+ méthode 1 (paramètres) + méthode 2 (paramètres) ...

## Exemple

Compte
+ numéro: entier + nom: chaîne ...
+ débiter(montant: réel) + créditer(montant: réel) ...



## Diagramme de classes (2/5)

---

- Une classe peut être en relation avec d'autres
- Un objet d'une classe peut manipuler un objet d'une autre classe
  - Relation d'association
  - Exemple: un client utilise un distributeur
- Un objet d'une classe peut contenir un objet d'une autre classe
  - Relation d'agrégation
  - Exemple: un client possède un ou plusieurs comptes
- Une classe peut représenter un sous-ensemble d'objets d'une autre classe qui sont plus spécifiques
  - Relation d'héritage
  - Exemple: les guichetiers sont une catégorie de personnels





# Diagramme de classes (3/5)

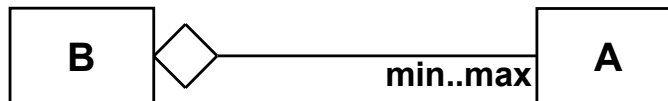
- Relation d'association

- Une classe B est associée à une classe A



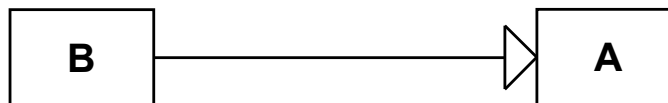
- Relation d'agrégation

- Une classe B possède des objets d'une classe A



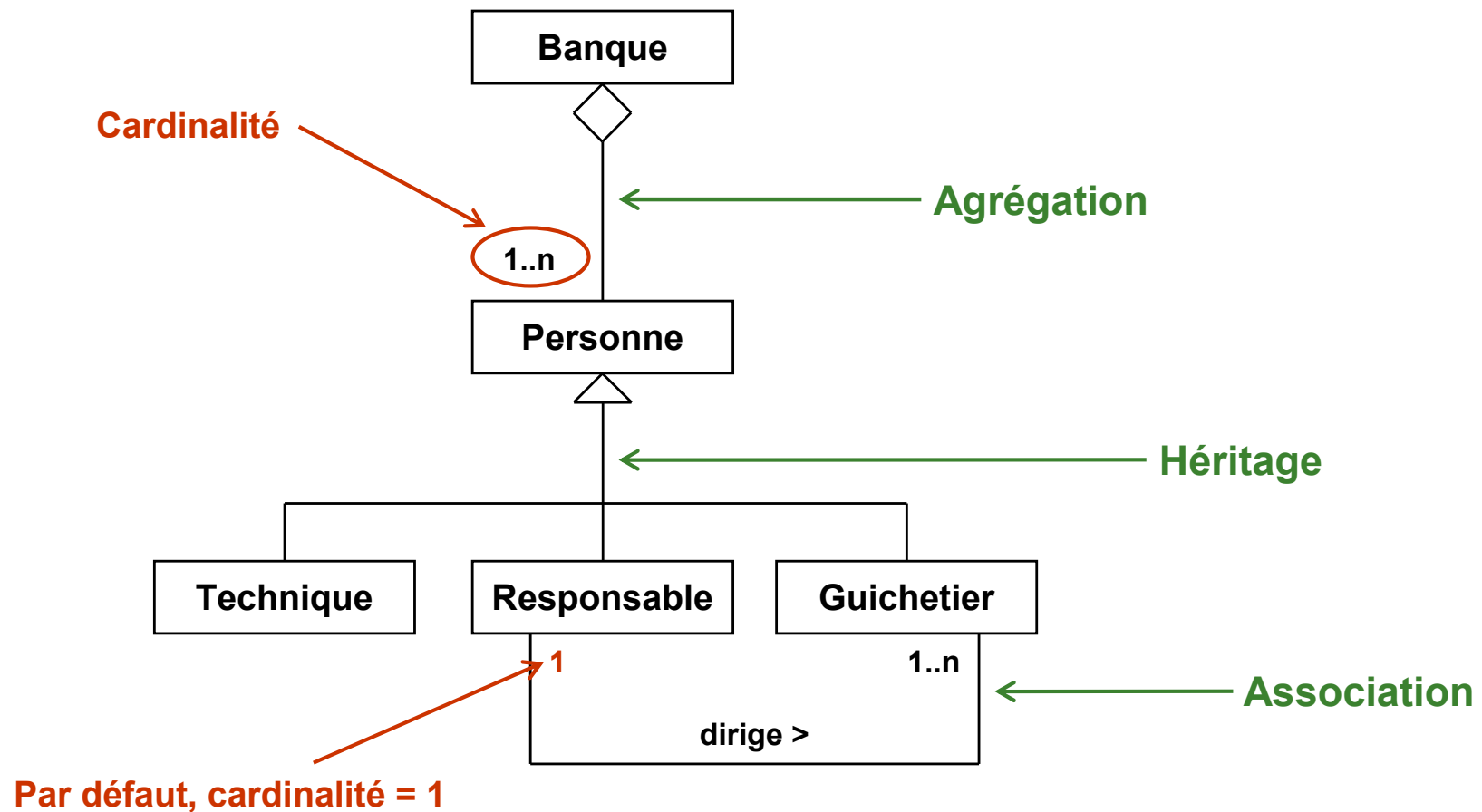
- Relation d'héritage

- Une classe B hérite des caractéristiques d'une classe A



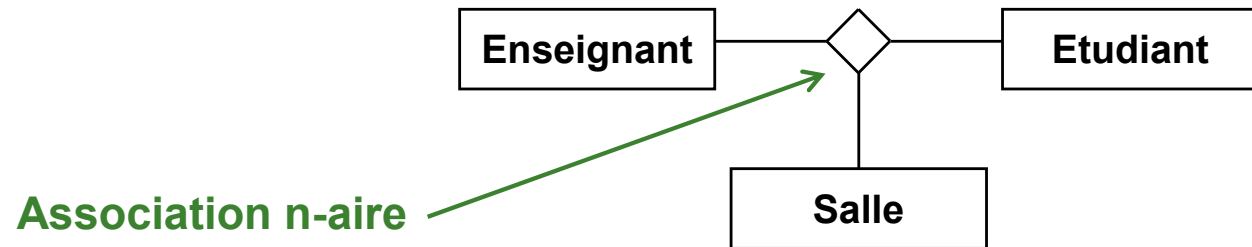
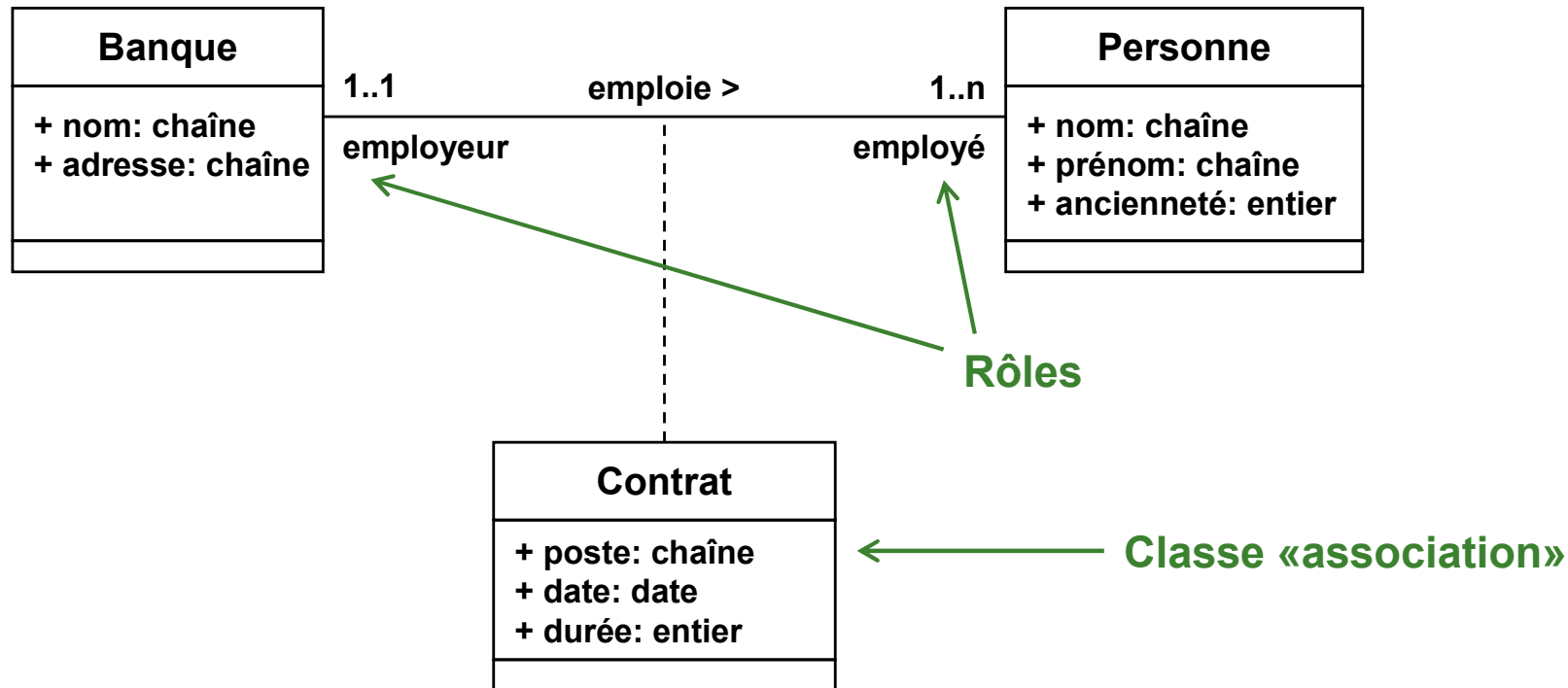


# Diagramme de classes (4/5)





# Diagramme de classes (5/5)





## 2<sup>ème</sup> étape: structure de la base de données

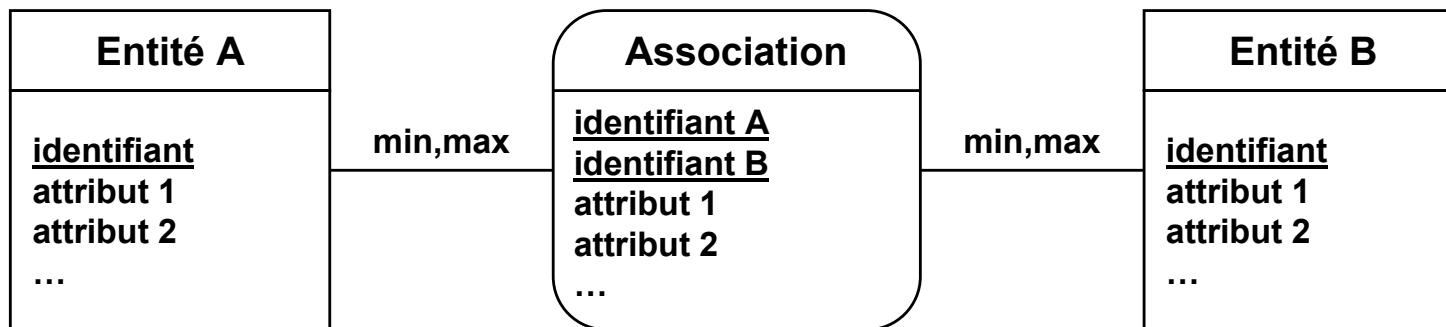
---

- Identifier les données à gérer dans une base
- Chaque catégorie de données  $\Rightarrow$  une table
  - Attributs
    - Type de l'information
    - Domaine de valeurs
  - Clés primaires
- Relation entre données  $\Rightarrow$  table «association» ?
  - Attributs
  - Cardinalité
- Utilisation du modèle entité-association



# Modèle de données entité-association (1/5)

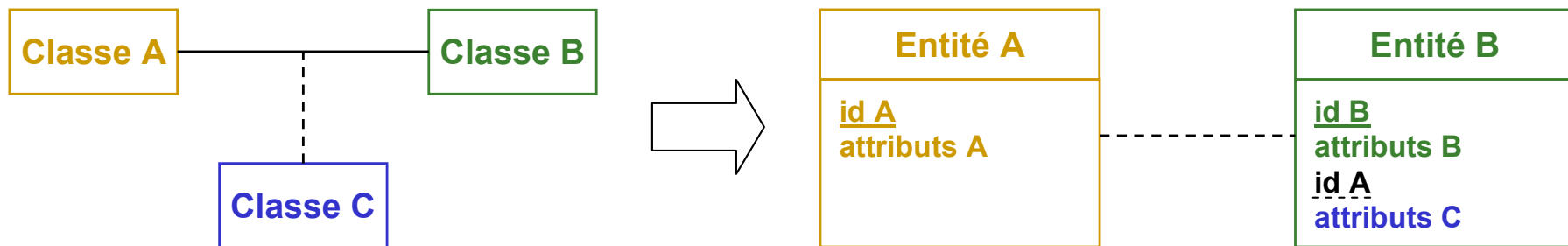
- Construction du modèle entité-association à partir du diagramme de classes
- Entité = table
  - Equivalent d'une classe «normale»
- Association = table qui relie plusieurs tables
  - Equivalent d'une classe «association»





## Modèle de données entité-association (2/5)

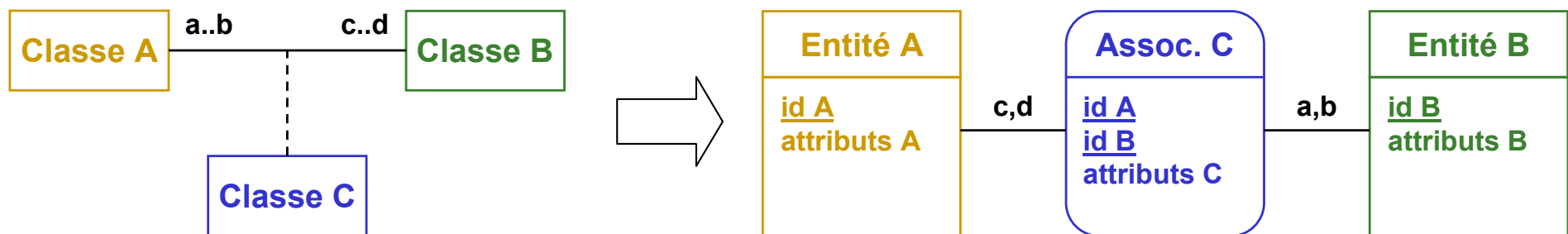
- Diagramme de classe  $\Rightarrow$  modèle de données entité-association
- Classe «normale»  $\Rightarrow$  entité
  - ❑ Attributs classe  $\Rightarrow$  attributs entité
  - ❑ Identifier clé primaire
- Relation d'association 1-1 ( $1..1 \Leftrightarrow 1..1$ )
  - ❑ Inclure clé primaire de A comme clé étrangère de B (ou inverse)
  - ❑ Attributs classe «association» avec clé étrangère
  - ❑ Rappel: clé étrangère = clé primaire d'une autre entité





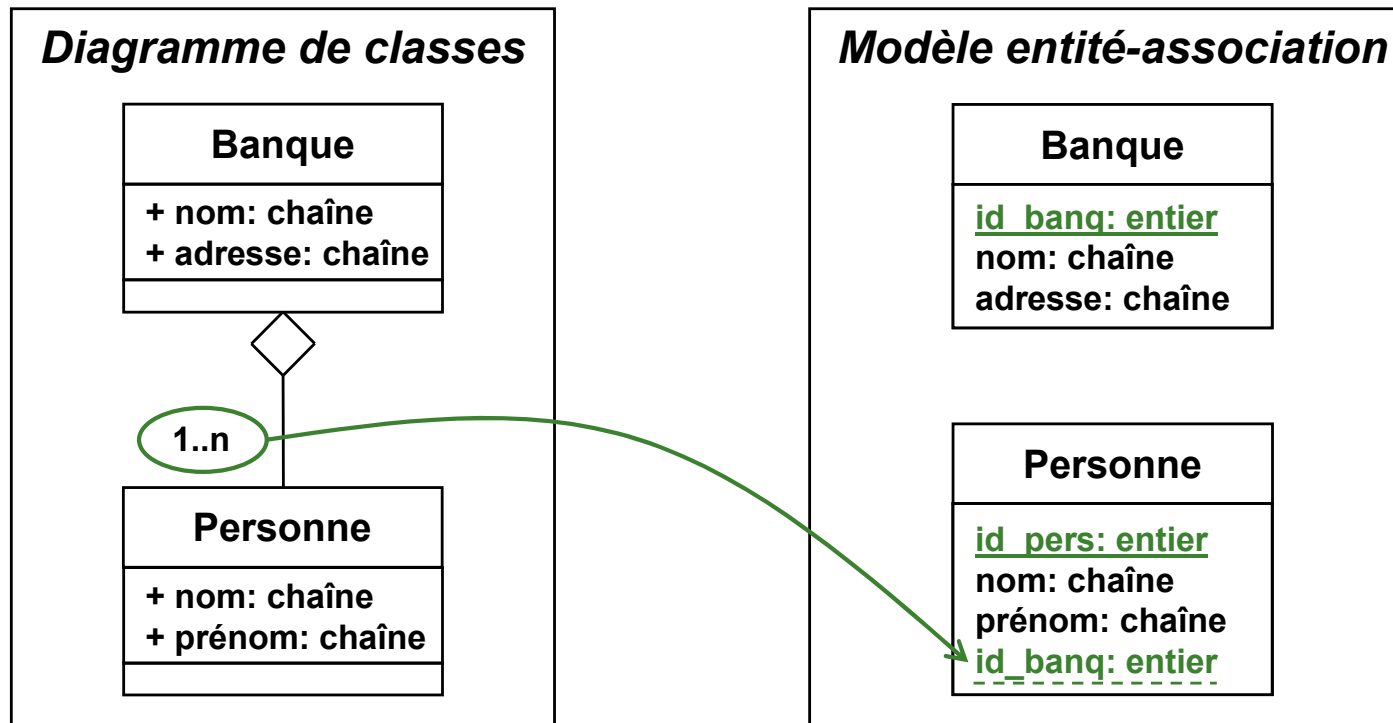
# Modèle de données entité-association (3/5)

- Association 1-N ( $1..1 \Leftrightarrow m..n$ )
  - ❑ Comme association 1-1, mais un seul sens possible
  - ❑ Inclure clé primaire de A comme clé étrangère de B
  - ❑ Attributs classe «association» dans B
- Association M-N ( $m..n \Leftrightarrow m..n$ )
  - ❑ Nouvelle association relie les clés primaires de A et de B
  - ❑ Clé primaire association = clé primaire de A + clé primaire de B
  - ❑ Attributs classe «association» dans nouvelle association





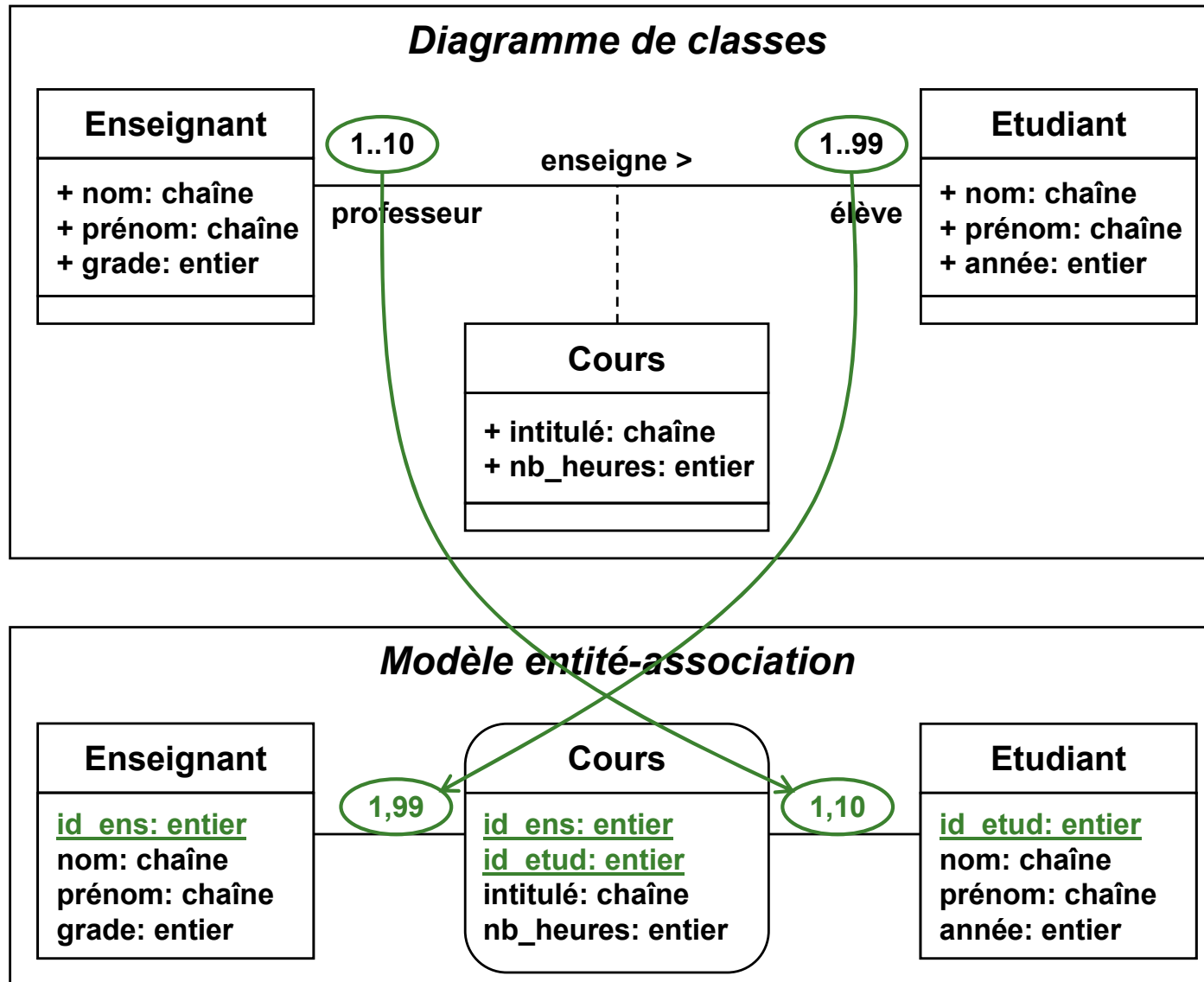
# Modèle de données entité-association (4/5)







# Modèle de données entité-association (5/5)





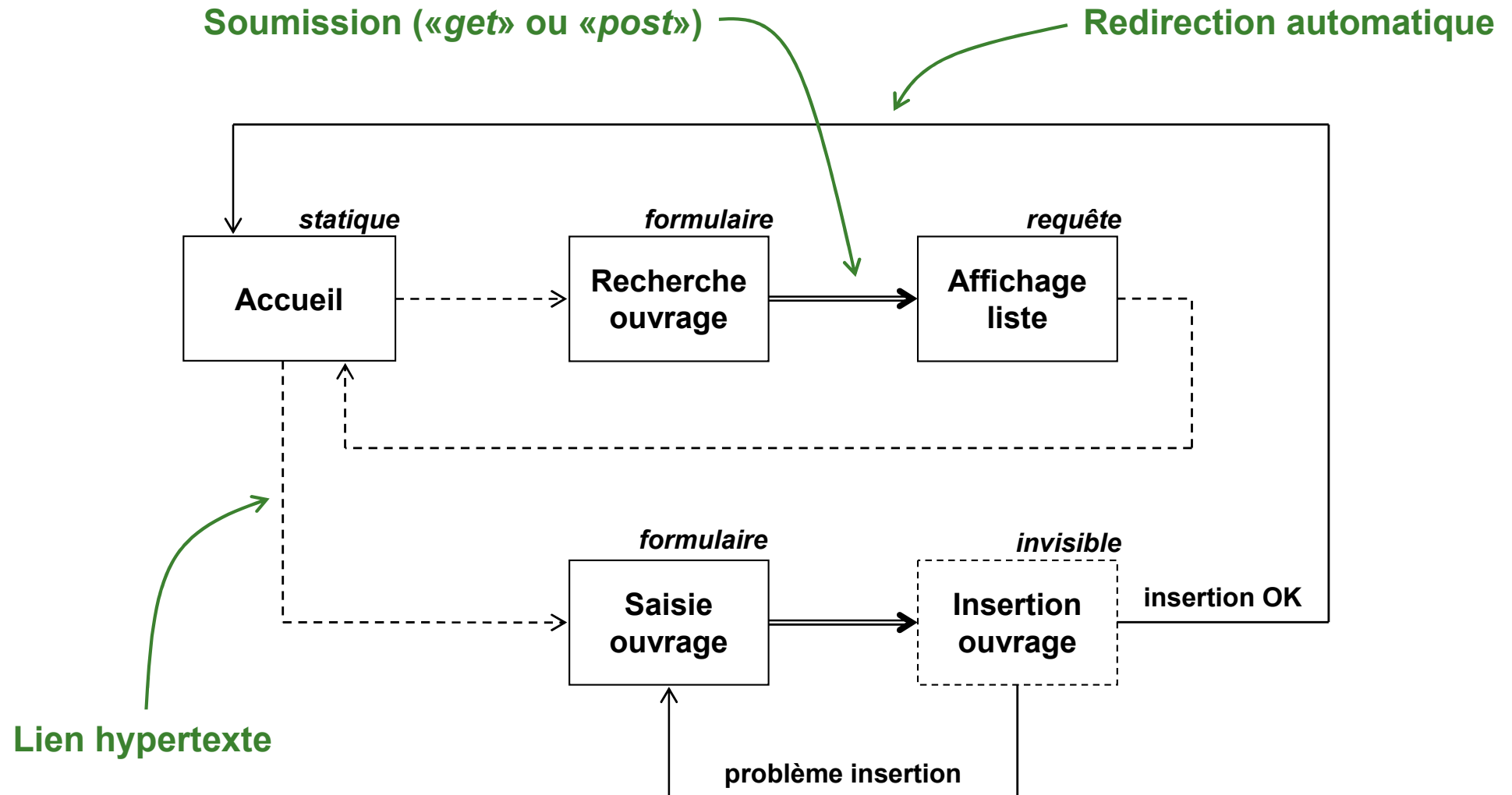
## 3<sup>ème</sup> étape: interface du site

---

- Identifier les pages nécessaires
  - ❑ Pages «formulaire»: interface de saisie
    - Définir les champs
    - Définir les contraintes
  - ❑ Pages «requête»: affichage du contenu
    - Définir le type d'affichage
    - Définir les requêtes
    - Définir des index pour les tables
  - ❑ Pages «invisibles»: gestion de la base
    - Définir les requêtes
    - Définir les contraintes
  - ❑ Pages statiques: aucun contenu en base
- Définir l'enchaînement des pages
  - ❑ Lien hypertexte
  - ❑ Soumission d'un formulaire
  - ❑ Redirection automatique



# Exemple de schéma de navigation





# Consignes pour l'esquisse des pages

---

- Apparence de chaque page
  - ❑ Disposition des éléments
  - ❑ Charte graphique
  
- Requêtes
  - ❑ Code SQL nécessaire
  - ❑ Contraintes (tests à effectuer)
  
- Formulaires
  - ❑ Champs (nom, type, valeur par défaut)
  - ❑ Contraintes (tests avant envoi)
  - ❑ Méthode d'envoi



# PARTIE VII

## Compléments de PHP

**Copyright (c) 1999-2011 - Bruno Bachelet - [bruno@nawouak.net](mailto:bruno@nawouak.net) - <http://www.nawouak.net>**

La permission est accordée de copier, distribuer et/ou modifier ce document sous les termes de la licence *GNU Free Documentation License*, Version 1.1 ou toute version ultérieure publiée par la fondation *Free Software Foundation*. Voir cette licence pour plus de détails (<http://www.gnu.org>).



# Générer autre chose que du HTML

---

- PHP peut générer autre chose que du HTML
- Une image (GIF, JPEG, PNG...)
  - ❑ Bibliothèque GD2
  - ❑ Permet de lire des images
  - ❑ Permet de créer ses propres images
  - ❑ Exemple: compteur de visite
    - ``
- Un document PDF
  - ❑ Bibliothèques PDFLib et FPDF
  - ❑ Permet de créer ses propres PDF
  - ❑ Exemple: générer une facture pour un client
- Nécessité d'ajouter des modules à PHP
  - ❑ Modifier le fichier «`php.ini`»
  - ❑ Ne sont pas toujours installés



# Générer une image avec PHP

---

- Extrait d'un code PHP pour compteur de visite  
`header("Content-type: image/png");`

```
$requete = "SELECT * FROM global WHERE (name='counter')";
$result = @mysql_query($requete);
$enr = @mysql_fetch_object($result);
$compteur = $enr->value;
```

```
$image = imageCreate(34,7);
$noir = imageColorAllocate($image,0,0,0);
$blanc = imageColorAllocate($image,255,255,255);
imageFilledRectangle($image,0,0,34,7,$blanc);
imageString($image,1,0,0,$compteur,$noir);
imagePNG($image);
imageDestroy($image);
```



# Générer un document PDF avec PHP

---

- Extrait d'un code PHP pour «*Hello World*»

```
header("Content-Type: application/pdf");
header("Content-Disposition: filename=hello.pdf");
```

```
$pdf = PDF_new();
pdf_begin_page_ext($pdf, 595, 842, "");
```

```
$fonte = pdf_load_font($pdf, "Helvetica-Bold", "winansi", "");
pdf_setfont($pdf, $fonte, 24.0);
```

```
pdf_set_text_pos($pdf, 50, 700);
pdf_show($pdf, "Hello world !");
```

```
pdf_end_page_ext($pdf, "");
pdf_end_document($pdf, "");
```

```
$buffer = pdf_get_buffer($pdf);
echo $buffer;
```





# Envoi d'email par PHP

---

- Peut être utile pour les bulletins d'information (ou «*newsletters*»)
- `boolean mail(string $destinataire, string $sujet, string $message, string $entetes)`
- **\$entetes**: entêtes du message
  - ❑ Adresse de l'expéditeur
  - ❑ Adresse de retour
  - ❑ Type de contenu
- Certains fournisseurs désactivent cette fonctionnalité
  - ❑ Permet d'éviter certaines formes de spam
- D'autres proposent leur propre fonction
  - ❑ Mécanisme similaire



# Construction des messages

---

- Message = entêtes + corps
- Format d'un entête
  - *nom\_entete* : *contenu\_entete*
  - Attention à la casse dans le nom des entêtes

- Exemple

```
<?php
```

```
$entetes = "From: bruno@nawouak.net\n";
$entetes .= "Reply-To: bruno@nawouak.net\n";
```

```
$message = "Salut !\n\n";
$message .= "Je teste l'envoi de mail par PHP.\n";
$message .= "Confirme-moi que ça fonctionne.\n\n";
$message .= "Ciao !";
```

```
mail("copain@gmail.com", "Test PHP", $message, $entetes);
?>
```



# Accès aux fichiers sur le serveur

---

- PHP peut accéder à des fichiers sur le serveur
  - ❑ Accès uniquement aux fichiers dans la zone HTTP
    - Fichiers accessibles par URL
  - ❑ Accès aux fichiers protégés par fichier « **.htaccess** »
    - Fichier « **.htaccess** » limite l'accès par HTTP
  
- Manipulations possibles par PHP
  - ❑ Consultation
    - Liste des fichiers d'un répertoire
    - Contenu d'un fichier
  - ❑ Modification
    - Renommage d'un fichier
    - Modification du contenu d'un fichier
  - ❑ Suppression fichier et répertoire
  - ❑ Création fichier et répertoire



# Consultation d'un répertoire

- Ouverture du répertoire
  - `$dossier = opendir($chemin);`
- Parcours des éléments du répertoire
  - `while ($element = readdir($dossier)) ...`
- Type des éléments
  - Fichier: `if (is_file($element)) ...`
  - Répertoire: `if (is_dir($element)) ...`
- Extrait du code PHP d'un diaporama

```
$chemin = "photos/";
$dossier = opendir($chemin);

while ($fichier = readdir($dossier)) {
 if (is_file($chemin.$fichier)) {
 echo "<p></p>";
 }
}

closedir($dossier);
```



# Manipulation des fichiers

---

- Copier un fichier
  - `copy($chemin_original, $chemin_copie);`
- Renommer / déplacer un fichier
  - `rename($chemin_ancien, $chemin_nouveau);`
- Suppression d'un fichier
  - `unlink($chemin);`
- Créer un répertoire
  - `mkdir($chemin, $droits);`
  - Droits d'accès: `$droits = "0700"`
- Détruire un répertoire vide
  - `rmdir($chemin);`



# Fichiers binaires en base de données

---

- Avantages
  - ❑ Pas de problèmes avec les droits des fichiers
  - ❑ Sauvegarde des données facilitée
  - ❑ Plus de risque d'incohérence entre les fichiers et la base
  - ❑ Données protégées: elles ne sont pas accessibles par URL
  
- Inconvénients
  - ❑ Taille importante de la base de données
  - ❑ Ralentissement de la base de données
  - ❑ Impossible d'accéder aux fichiers directement par URL
  - ❑ Attention à la requête «**SELECT \* from image**»
  
- Attribut de type «**BLOB**»
  - ❑ *Binary Large Object*
  - ❑ **BLOB, TINYBLOB, MEDIUMBLOB, LONGBLOB**



# Récupération d'une image dans une table

---

- Table «*photo*»
  - «*id*»: numéro de la photo
  - «*png*»: contenu binaire de la photo

- Extrait du code PHP

```
$id = @$_GET["id"];
```

```
$requete = "SELECT * FROM photo WHERE id='$id'";
```

```
$result = @mysql_query($requete);
```

```
$enr = @mysql_fetch_object($result);
```

```
if ($enr) {
```

```
 header("Content-type: image/png");
```

```
 echo $enr->png;
```

```
}
```



# Dépôt («*upload*») d'un fichier sur le serveur

---

- Formulaire
  - ❑ Méthode «*post*»
  - ❑ Encodage «**multipart/form-data**» (attribut «**enctype**»)
  - ❑ Insérer un champ de type «**file**»
    - **<input type="file" name="logo"/>**
- A la soumission du formulaire
  - ❑ Transmission du fichier au serveur
  - ❑ Stockage du fichier dans une zone temporaire
  - ❑ Le fichier porte un nom différent de celui d'origine
- PHP: accès au fichier par le tableau «**\_\_FILES**»
  - ❑ Chemin d'origine du fichier
    - **\$\_FILES["logo"]["name"]**
  - ❑ Chemin de la copie sur le serveur
    - **\$\_FILES["logo"]["tmp\_name"]**
  - ❑ Taille du fichier
    - **\$\_FILES["logo"]["size"]**